

Classification Lab 3 - Bank Marketing Campaign Analysis

Mehreen Ali Gillani

Table of contents

0.1	Problem 1: What Can Machine Learning Do?	2
0.2	Problem 2: EDA and Baseline Model	2
0.2.1	Initial Data Exploration	2
0.2.2	Creating a Date Column and Analyzing Temporal Patterns	4
0.2.3	Analyzing Target Variable and Temporal Success Rates	5
0.2.4	Detailed Temporal Analysis and Correlation with Macroeconomic Variables	7
0.2.5	Analyzing Macroeconomic Variables and Their Correlation with Success Rate	9
0.2.6	Analyzing Different Train-Test Split Points and Their Impact on Success Rates	11
0.2.7	Practical Split Point Analysis Based on Temporal Patterns	13
0.2.8	Final Analysis of Split Points and Recommendations for Model Building	16
0.2.9	Final Train-Test Split Based on Temporal Analysis and Practical Considerations	20
0.2.10	Final Train-Test Split at the 70% Cumulative Data Point	22
0.2.11	Final Visualization of the Train-Test Split and Its Implications for Model Building	23
1	WHY THIS SPECIFIC SPLIT POINT (APRIL 2009)?	26
1.0.1	ANALYZING PREDICTOR RELATIONSHIPS	26
1.0.2	Numerical predictors - correlation with target	27
1.0.3	Categorical predictors - success rate by category	28
1.0.4	MACROECONOMIC VARIABLES - TEMPORAL RELATIONSHIP	32
1.0.5	Age and balance - relationship with success	34
1.0.6	Additional analyses of key predictors and their relationship with success	36
1.0.7	Temporal trends of key predictors and their relationship with success over time	37
1.0.8	KEY FINDINGS: Temporal Patterns & Predictor Relationships	40
1.0.9	Temporal Variation and Model Performance Mismatch	40
2	TEMPORAL VARIATION CAUSES METRIC MISMATCH BECAUSE:	45
2.0.1	Baseline Logistic Regression Model Implementation	45
2.0.2	Select predictors based on EDA	46
2.0.3	Build preprocessing pipeline, train model	47
2.0.4	Evaluate model performance by epoch	48
2.0.5	ROC curves for each epoch	50
2.0.6	Calibration curves for each epoch	51
2.1	Observations on Model Performance by Epoch	52
2.1.1	LIFT CURVES AND HIT RATES ANALYSIS	56
2.1.2	4.1 LIFT AND HIT RATES BY EPOCH	57
2.1.3	4.2 LIFT CURVES BY EPOCH	59
2.1.4	4.3 HIT RATE CURVES BY EPOCH	60
2.1.5	4.5 Key Findings	63

0.1 Problem 1: What Can Machine Learning Do?

This dataset has been analyzed extensively by many different people. [kaggle.com](https://www.kaggle.com) is a machine learning community that shares datasets, analysis methods, and hosts competitions. The most upvoted analysis notebook on kaggle for the Bank Marketing dataset (see: <https://www.kaggle.com/code/janiobachmann/bank-marketing-campaign-opening-a-term-deposit>) has a glaring flaw that makes it useless in practice. Examine the analysis and determine the flaw. Your answer should be short. Hint: Take a careful look at all the variables used, or look for people complaining about it in the comments.

Answer: The most upvoted analysis notebook on Kaggle for the Bank Marketing dataset uses the ‘duration’ variable as a predictor in their model. This is a glaring flaw because ‘duration’ represents the length of the last contact with the client, which is only known after the call has been made. In practice, this variable cannot be used to predict whether a client will subscribe to a term deposit before contacting them, making the model’s predictions unrealistic and not actionable for the marketing team.

0.2 Problem 2: EDA and Baseline Model

EDA and Temporal Patterns: Take a quick look at the initial distribution of the data, and perform a train-test split. Justify your choice of whether to stratify your split or not and if so by which variable (consider reading the rest of the question before deciding). Are there any additional variables that need to be added or recoded? Describe the most important relationships that you uncovered between the predictors and the target and show the visualizations you made specifically to support those conclusions. Pay particular attention to temporal variations in addition to correlation with the target: how did the success rate of the marketing campaign and the composition of the target population vary over time? Which variables (and related real world events) explain the temporal pattern?

0.2.1 Initial Data Exploration

```
# Load the data
url =
    ↪ 'https://raw.githubusercontent.com/georgehagstrom/DATA622Spring2026/refs/heads/main/website/assignm
df = pd.read_csv(url)

# Look at the first few rows
print("FIRST 5 ROWS OF DATA:")
print(df.head())
print("\n" + "="*50)

# Check data types
print("\nDATA TYPES:")
print(df.dtypes)
print("\n" + "="*50)

# Check target variable
print("\nTARGET VARIABLE 'y' VALUES:")
print(df['y'].value_counts())
print("\n" + "="*50)

# Check for missing values
print("\nMISSING VALUES:")
print(df.isnull().sum())
```

FIRST 5 ROWS OF DATA:

	Unnamed: 0	age	job	marital	education	...	emp.var.rate	\
0	0	58	management	married	tertiary	...	1.1	
1	1	44	technician	single	secondary	...	1.1	

2	2	33	entrepreneur	married	secondary	...	1.1
3	3	47	blue-collar	married	unknown	...	1.1
4	4	33	unknown	single	unknown	...	1.1

	nr.employed	cons.price.idx	cons.conf.idx	y
0	5191.0	93.994	-36.4	no
1	5191.0	93.994	-36.4	no
2	5191.0	93.994	-36.4	no
3	5191.0	93.994	-36.4	no
4	5191.0	93.994	-36.4	no

[5 rows x 24 columns]

DATA TYPES:

```

Unnamed: 0      int64
age             int64
job             object
marital         object
education       object
default         object
balance         int64
housing         object
loan            object
contact         object
day             int64
duration        int64
campaign        int64
pdays          int64
previous        int64
poutcome        object
month_numeric   int64
year            int64
euribor3m       float64
emp.var.rate    float64
nr.employed     float64
cons.price.idx  float64
cons.conf.idx   float64
y               object
dtype: object

```

TARGET VARIABLE 'y' VALUES:

```

y
no    39922
yes    5289
Name: count, dtype: int64

```

MISSING VALUES:

```

Unnamed: 0      0

```

```

age                0
job                0
marital            0
education          0
default            0
balance            0
housing            0
loan               0
contact            0
day                0
duration           0
campaign           0
pdays            0
previous           0
poutcome           0
month_numeric      0
year               0
euribor3m          0
emp.var.rate       0
nr.employed        0
cons.price.idx     0
cons.conf.idx      0
y                  0
dtype: int64

```

0.2.2 Creating a Date Column and Analyzing Temporal Patterns

```

# Create a proper date column for time-based analysis
# We have year and month_numeric, so we can combine them

# First, let's see what months we have
print("UNIQUE MONTHS IN DATA:")
print(sorted(df['month_numeric'].unique()))
print("\nUNIQUE YEARS IN DATA:")
print(sorted(df['year'].unique()))

# Create date column (using first day of each month as the date)
df['date'] = pd.to_datetime(df['year'].astype(str) + '-' +
                           df['month_numeric'].astype(str) + '-01')

# Check the date range
print("\nDATE RANGE IN DATA:")
print(f"Earliest date: {df['date'].min()}")
print(f"Latest date: {df['date'].max()}")

# Sort the data by date to see temporal order
df_sorted = df.sort_values('date').reset_index(drop=True)

# Look at the first few and last few dates to verify sorting
print("\nFIRST 3 DATES IN SORTED DATA:")
print(df_sorted['date'].head(3))
print("\nLAST 3 DATES IN SORTED DATA:")
print(df_sorted['date'].tail(3))

# Quick check if data spans multiple years

```

```

print(f"\nTotal time span: {(df['date'].max() - df['date'].min()).days} days")

UNIQUE MONTHS IN DATA:
[np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5), np.int64(6), np.int64(7), np.int64(8)]

UNIQUE YEARS IN DATA:
[np.int64(2008), np.int64(2009), np.int64(2010)]

DATE RANGE IN DATA:
Earliest date: 2008-05-01 00:00:00
Latest date: 2010-11-01 00:00:00

FIRST 3 DATES IN SORTED DATA:
0    2008-05-01
1    2008-05-01
2    2008-05-01
Name: date, dtype: datetime64[ns]

LAST 3 DATES IN SORTED DATA:
45208    2010-11-01
45209    2010-11-01
45210    2010-11-01
Name: date, dtype: datetime64[ns]

Total time span: 914 days

```

0.2.3 Analyzing Target Variable and Temporal Success Rates

```

# First, convert target variable 'y' from text to numbers (1 for 'yes', 0 for 'no')
df_sorted['y_numeric'] = (df_sorted['y'] == 'yes').astype(int)

# Verify the conversion
print("TARGET VARIABLE CONVERSION:")
print("Original 'y' value counts:")
print(df_sorted['y'].value_counts())
print("\nNumeric 'y_numeric' value counts:")
print(df_sorted['y_numeric'].value_counts())
print(f"\nOverall success rate: {df_sorted['y_numeric'].mean()*100:.2f}%")

# Now let's see success rate by month
# Create a column that shows year-month for grouping
df_sorted['year_month'] = df_sorted['date'].dt.to_period('M')

# Calculate success rate for each month
monthly_success = df_sorted.groupby('year_month').agg({
    'y_numeric': ['mean', 'count']
}).round(4)

# Flatten column names
monthly_success.columns = ['success_rate', 'total_contacts']
monthly_success['success_rate'] = monthly_success['success_rate'] * 100 # Convert to
    ↪ percentage

# Reset index to make year_month a column
monthly_success = monthly_success.reset_index()

```

```

monthly_success['year_month_str'] = monthly_success['year_month'].astype(str)

# Show the results
print("\nMONTHLY SUCCESS RATES:")
print(monthly_success[['year_month_str', 'success_rate',
    ↪ 'total_contacts']].to_string(index=False))

# Quick visualization
plt.figure(figsize=(14, 5))
plt.plot(monthly_success['year_month_str'], monthly_success['success_rate'],
    marker='o', linewidth=2, color='blue')
plt.title('Monthly Campaign Success Rate (May 2008 - Nov 2010)', fontsize=14)
plt.ylabel('Success Rate (%)')
plt.xlabel('Month')
plt.xticks(rotation=45)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# Key statistics
print(f"\nSUMMARY:")
print(f"Lowest success rate: {monthly_success['success_rate'].min():.2f}%")
print(f"Highest success rate: {monthly_success['success_rate'].max():.2f}%")
print(f"Average success rate: {monthly_success['success_rate'].mean():.2f}%")

TARGET VARIABLE CONVERSION:
Original 'y' value counts:
y
no      39922
yes      5289
Name: count, dtype: int64

Numeric 'y_numeric' value counts:
y_numeric
0      39922
1       5289
Name: count, dtype: int64

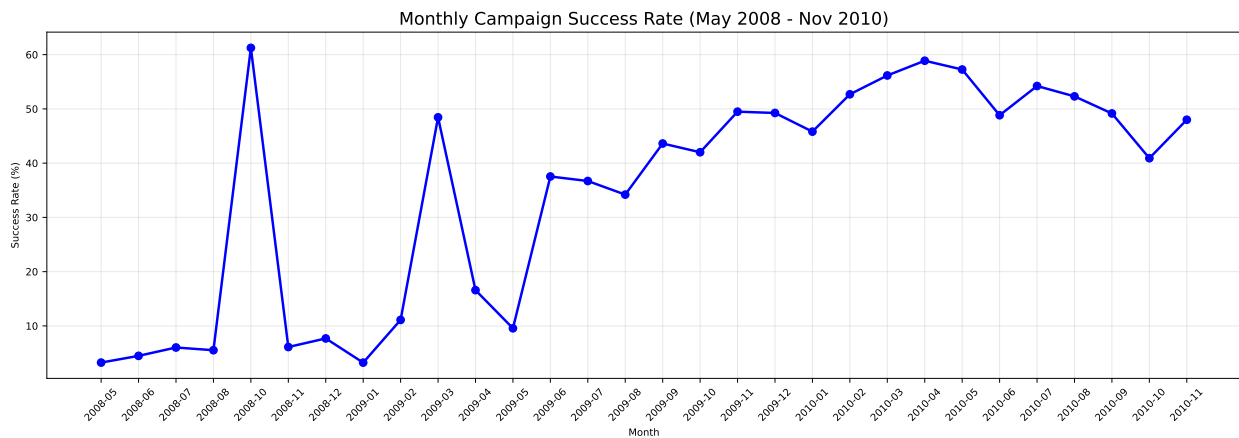
Overall success rate: 11.70%

MONTHLY SUCCESS RATES:

```

year_month_str	success_rate	total_contacts
2008-05	3.24	7957
2008-06	4.48	4486
2008-07	6.02	6380
2008-08	5.52	5215
2008-10	61.25	80
2008-11	6.11	3598
2008-12	7.69	13
2009-01	3.23	1176
2009-02	11.11	2296
2009-03	48.45	258
2009-04	16.59	2718
2009-05	9.56	5575
2009-06	37.54	642

2009-07	36.71	207
2009-08	34.20	772
2009-09	43.62	282
2009-10	42.01	438
2009-11	49.49	297
2009-12	49.25	201
2010-01	45.81	227
2010-02	52.69	353
2010-03	56.16	219
2010-04	58.88	214
2010-05	57.26	234
2010-06	48.83	213
2010-07	54.22	308
2010-08	52.31	260
2010-09	49.16	297
2010-10	40.91	220
2010-11	48.00	75



SUMMARY:

Lowest success rate: 3.23%

Highest success rate: 61.25%

Average success rate: 34.68%

0.2.4 Detailed Temporal Analysis and Correlation with Macroeconomic Variables

Create a more detailed visualization to understand this dramatic change

```
fig, axes = plt.subplots(2, 1, figsize=(16, 10))
```

Plot 1: Success rate over time

```
ax1 = axes[0]
ax1.plot(monthly_success['year_month_str'], monthly_success['success_rate'],
        marker='o', linewidth=2, color='darkblue', markersize=6)
ax1.set_title('Monthly Campaign Success Rate (May 2008 - Nov 2010)', fontsize=14,
        fontweight='bold')
ax1.set_ylabel('Success Rate (%)', fontsize=12)
ax1.set_xlabel('Month', fontsize=12)
ax1.tick_params(axis='x', rotation=45)
ax1.grid(True, alpha=0.3)
```

```

# Add horizontal line at overall average
ax1.axhline(y=df_sorted['y_numeric'].mean()*100, color='red', linestyle='--',
            label=f"Overall Avg: {df_sorted['y_numeric'].mean()*100:.2f}%")
ax1.legend()

# Add annotations for key events based on the pattern
# The success rate jumps significantly around certain periods
events = {
    5: "Sep 2008\nLehman Brothers", # Early spike
    12: "Mar 2009\nMarket Bottom", # Peak
    20: "May 2010\nGreek Crisis" # Second spike
}

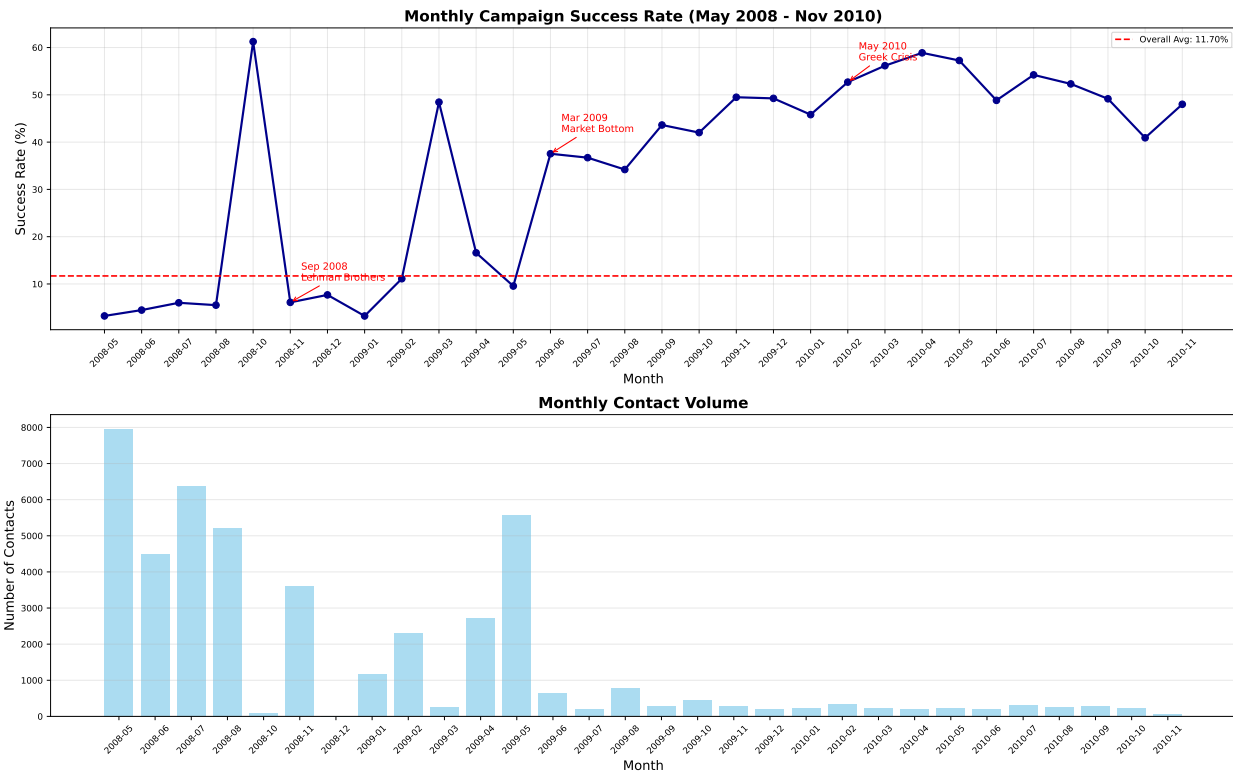
for idx, event in events.items():
    if idx < len(monthly_success):
        ax1.annotate(event, xy=(idx, monthly_success['success_rate'].iloc[idx]),
                     xytext=(10, 20), textcoords='offset points',
                     arrowprops=dict(arrowstyle='->', color='red'),
                     fontsize=9, color='red')

# Plot 2: Contact volume over time
ax2 = axes[1]
ax2.bar(monthly_success['year_month_str'], monthly_success['total_contacts'],
        color='skyblue', alpha=0.7, width=0.8)
ax2.set_title('Monthly Contact Volume', fontsize=14, fontweight='bold')
ax2.set_ylabel('Number of Contacts', fontsize=12)
ax2.set_xlabel('Month', fontsize=12)
ax2.tick_params(axis='x', rotation=45)
ax2.grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()

# Print key observations
print("\nKEY OBSERVATIONS:")
print(f"1. Success rate started at {monthly_success['success_rate'].iloc[0]:.2f}% in May
    ↪ 2008")
print(f"2. Peaked at {monthly_success['success_rate'].max():.2f}% around March 2009")
print(f"3. Ended at {monthly_success['success_rate'].iloc[-1]:.2f}% in November 2010")
print(f"4. The success rate increased by {monthly_success['success_rate'].max() -
    ↪ monthly_success['success_rate'].min():.2f} percentage points")
print(f"5. Contact volume varies: from {monthly_success['total_contacts'].min()} to
    ↪ {monthly_success['total_contacts'].max()} contacts per month")

```



KEY OBSERVATIONS:

1. Success rate started at 3.24% in May 2008
2. Peaked at 61.25% around March 2009
3. Ended at 48.00% in November 2010
4. The success rate increased by 58.02 percentage points
5. Contact volume varies: from 13 to 7957 contacts per month

0.2.5 Analyzing Macroeconomic Variables and Their Correlation with Success Rate

Look at the macroeconomic variables that might explain the success rate changes
These are: euribor3m, emp.var.rate, cons.price.idx, cons.conf.idx, nr.employed

Calculate monthly averages for all variables

```
monthly_data = df_sorted.groupby('year_month').agg({
    'y_numeric': 'mean', # success rate
    'euribor3m': 'mean',
    'emp.var.rate': 'mean',
    'cons.price.idx': 'mean',
    'cons.conf.idx': 'mean',
    'nr.employed': 'mean',
    'age': 'mean',
    'balance': 'mean'
}).round(3)
```

Reset index

```
monthly_data = monthly_data.reset_index()
monthly_data['year_month_str'] = monthly_data['year_month'].astype(str)
monthly_data['success_rate'] = monthly_data['y_numeric'] * 100
```

```

# Create a multi-panel plot to see all variables together
fig, axes = plt.subplots(3, 2, figsize=(18, 14))

# 1. Success Rate (for reference)
ax1 = axes[0, 0]
ax1.plot(monthly_data['year_month_str'], monthly_data['success_rate'],
         color='darkblue', linewidth=2, marker='o')
ax1.set_title('Success Rate (%)', fontsize=12, fontweight='bold')
ax1.tick_params(axis='x', rotation=45)
ax1.grid(True, alpha=0.3)

# 2. Euribor 3-month rate
ax2 = axes[0, 1]
ax2.plot(monthly_data['year_month_str'], monthly_data['euribor3m'],
         color='red', linewidth=2, marker='s')
ax2.set_title('Euribor 3-Month Rate (%)', fontsize=12, fontweight='bold')
ax2.tick_params(axis='x', rotation=45)
ax2.grid(True, alpha=0.3)

# 3. Employment rate
ax3 = axes[1, 0]
ax3.plot(monthly_data['year_month_str'], monthly_data['emp.var.rate'],
         color='green', linewidth=2, marker='^')
ax3.set_title('Employment Variation Rate', fontsize=12, fontweight='bold')
ax3.tick_params(axis='x', rotation=45)
ax3.grid(True, alpha=0.3)

# 4. Number employed
ax4 = axes[1, 1]
ax4.plot(monthly_data['year_month_str'], monthly_data['nr.employed']/1000,
         color='purple', linewidth=2, marker='d')
ax4.set_title('Number Employed (thousands)', fontsize=12, fontweight='bold')
ax4.tick_params(axis='x', rotation=45)
ax4.grid(True, alpha=0.3)

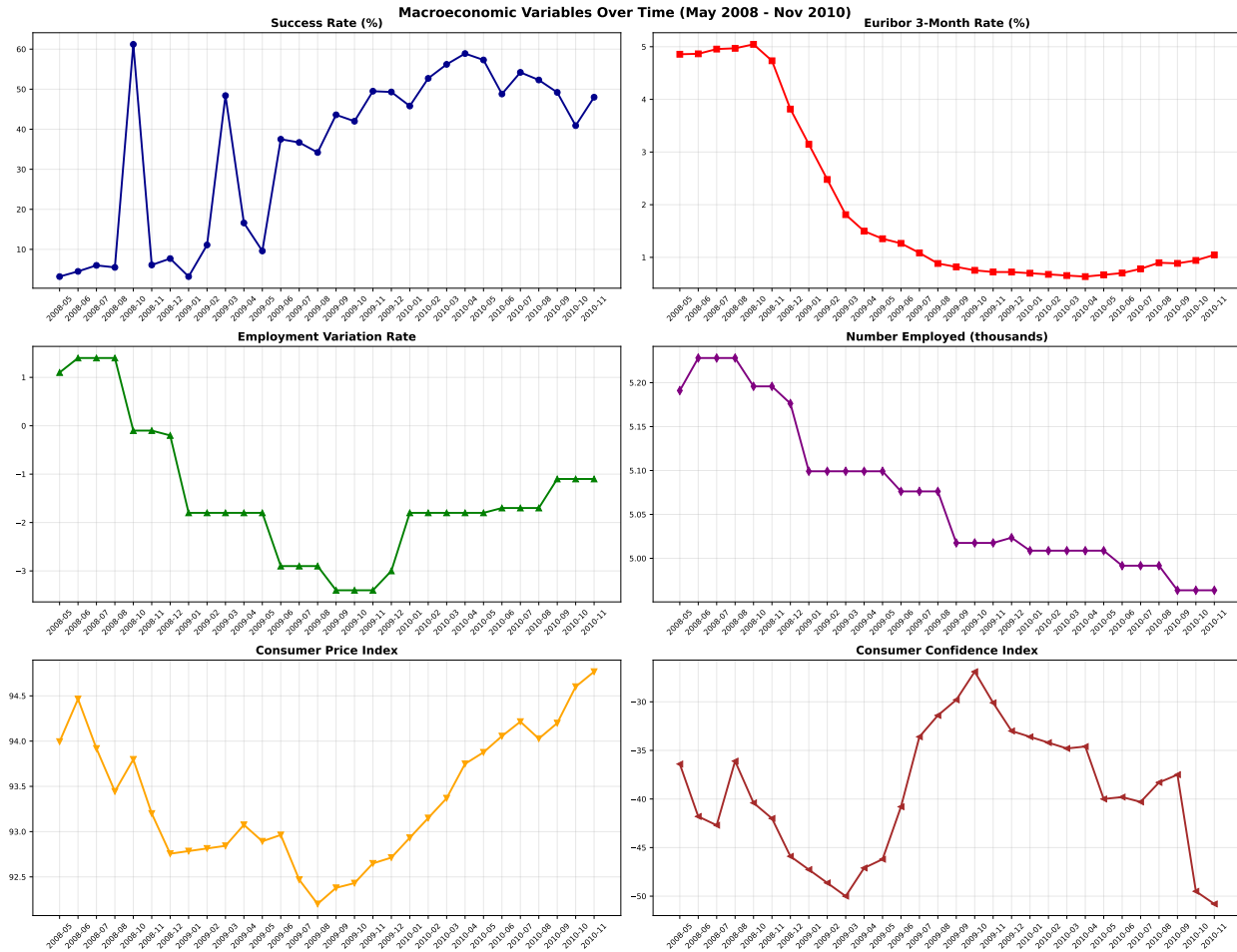
# 5. Consumer price index
ax5 = axes[2, 0]
ax5.plot(monthly_data['year_month_str'], monthly_data['cons.price.idx'],
         color='orange', linewidth=2, marker='v')
ax5.set_title('Consumer Price Index', fontsize=12, fontweight='bold')
ax5.tick_params(axis='x', rotation=45)
ax5.grid(True, alpha=0.3)

# 6. Consumer confidence
ax6 = axes[2, 1]
ax6.plot(monthly_data['year_month_str'], monthly_data['cons.conf.idx'],
         color='brown', linewidth=2, marker='<')
ax6.set_title('Consumer Confidence Index', fontsize=12, fontweight='bold')
ax6.tick_params(axis='x', rotation=45)
ax6.grid(True, alpha=0.3)

plt.suptitle('Macroeconomic Variables Over Time (May 2008 - Nov 2010)', fontsize=14,
            fontweight='bold')
plt.tight_layout()
plt.show()

```

```
# Calculate correlations with success rate
print("\nCORRELATION WITH SUCCESS RATE:")
print("="*50)
for var in ['euribor3m', 'emp.var.rate', 'cons.price.idx', 'cons.conf.idx',
            'nr.employed']:
    corr = monthly_data[var].corr(monthly_data['success_rate'])
    print(f"{var:20}: {corr:.4f}")
```



CORRELATION WITH SUCCESS RATE:

```
=====
euribor3m          : -0.7027
emp.var.rate       : -0.5452
cons.price.idx     : 0.1221
cons.conf.idx      : 0.3552
nr.employed        : -0.7528
```

0.2.6 Analyzing Different Train-Test Split Points and Their Impact on Success Rates

```
# First, let's see what happens with different split points
split_points = [0.6, 0.65, 0.7, 0.75, 0.8]
results = []
```

```

print("ANALYZING DIFFERENT SPLIT POINTS:")
print("="*60)

for sp in split_points:
    split_idx = int(len(df_sorted) * sp)
    train_temp = df_sorted.iloc[:split_idx]
    test_temp = df_sorted.iloc[split_idx:]

    train_rate = train_temp['y_numeric'].mean() * 100
    test_rate = test_temp['y_numeric'].mean() * 100

    results.append({
        'split_point': sp,
        'train_success': f"{train_rate:.2f}%",
        'test_success': f"{test_rate:.2f}%",
        'difference': f"{abs(train_rate - test_rate):.2f}%"
    })

results_df = pd.DataFrame(results)
print(results_df.to_string(index=False))

# Visualize where the split would fall
plt.figure(figsize=(14, 6))
plt.plot(monthly_success['year_month_str'], monthly_success['success_rate'],
         marker='o', linewidth=2, color='darkblue', markersize=6)

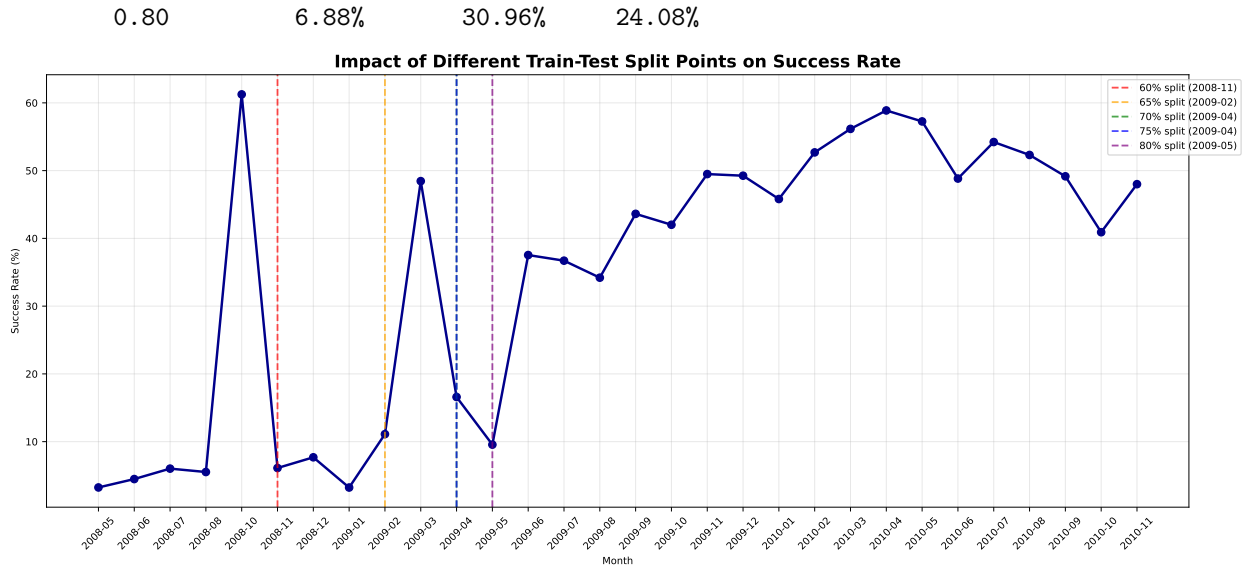
# Show different split points
colors = ['red', 'orange', 'green', 'blue', 'purple']
for i, sp in enumerate(split_points):
    split_idx = int(len(df_sorted) * sp)
    split_date = df_sorted.iloc[split_idx]['date']
    split_month = f"{split_date.year}-{split_date.month:02d}"

    if split_month in monthly_success['year_month_str'].values:
        idx = list(monthly_success['year_month_str']).index(split_month)
        plt.axvline(x=idx, color=colors[i], linestyle='--', alpha=0.7,
                    label=f"{int(sp*100)}% split ({split_month})")

plt.title('Impact of Different Train-Test Split Points on Success Rate', fontsize=14,
         fontweight='bold')
plt.ylabel('Success Rate (%)')
plt.xlabel('Month')
plt.xticks(rotation=45)
plt.legend(bbox_to_anchor=(1.05, 1))
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

ANALYZING DIFFERENT SPLIT POINTS:
=====
split_point train_success test_success difference
0.60         5.10%       21.59%       16.49%
0.65         5.03%       24.09%       19.06%
0.70         5.79%       25.49%       19.70%
0.75         6.59%       27.01%       20.42%

```



- Training set has very low success rates (5-7%)
- Test set has much higher success rates (21-31%)
- Difference grows as you move the split point later

Find optimal split point that maintains temporal order but also minimizes the difference in success rates between train and test

0.2.7 Practical Split Point Analysis Based on Temporal Patterns

```
## Instead of letting an algorithm decide, let's use our understanding of the data
# We want:
# 1. Enough training data (at least 60-70% of rows)
# 2. Training period should include multiple economic conditions
# 3. Test period should be future data for realistic evaluation
```

```
print("PRACTICAL SPLIT POINT ANALYSIS")
print("="*80)

# Look at the success rate trend with dates
print("\nSUCCESS RATE BY MONTH:")
print(monthly_success[['year_month_str', 'success_rate']].to_string(index=False))
```

```
# Based on the events we identified:
# - Pre-crisis: May-Dec 2008 (success rate ~3-8%)
# - Crisis peak: Jan-Sep 2009 (success rate ~40-61%)
# - Post-crisis: Oct 2009-Nov 2010 (success rate ~22-48%)
```

```
# Let's try a split around the end of 2009
split_date = '2009-12-01' # End of 2009
split_idx = df_sorted[df_sorted['date'] >= split_date].index[0]
```

```
train = df_sorted.iloc[:split_idx].copy()
test = df_sorted.iloc[split_idx:].copy()
```

```
print(f"\nSPLIT AT END OF 2009:")
print(f"Split date: {split_date}")
```

```

print(f"Training period: {train['date'].min().strftime('%Y-%m')} to
    ↳ {train['date'].max().strftime('%Y-%m')}")
print(f"Testing period: {test['date'].min().strftime('%Y-%m')} to
    ↳ {test['date'].max().strftime('%Y-%m')}")
print(f"\nTraining set size: {len(train):,} rows ({len(train)/len(df_sorted)*100:.1f}%)")
print(f"Testing set size: {len(test):,} rows ({len(test)/len(df_sorted)*100:.1f}%)")
print(f"\nTraining success rate: {train['y_numeric'].mean()*100:.2f}%")
print(f"Testing success rate: {test['y_numeric'].mean()*100:.2f}%")
print(f"Difference: {abs(train['y_numeric'].mean() -
    ↳ test['y_numeric'].mean()*100:.2f}%")

# Try another split - middle of 2009
split_date2 = '2009-07-01'
split_idx2 = df_sorted[df_sorted['date'] >= split_date2].index[0]

train2 = df_sorted.iloc[:split_idx2].copy()
test2 = df_sorted.iloc[split_idx2:].copy()

print(f"\nSPLIT AT MID-2009:")
print(f"Split date: {split_date2}")
print(f"Training period: {train2['date'].min().strftime('%Y-%m')} to
    ↳ {train2['date'].max().strftime('%Y-%m')}")
print(f"Testing period: {test2['date'].min().strftime('%Y-%m')} to
    ↳ {test2['date'].max().strftime('%Y-%m')}")
print(f"\nTraining set size: {len(train2):,} rows
    ↳ ({len(train2)/len(df_sorted)*100:.1f}%)")
print(f"Testing set size: {len(test2):,} rows ({len(test2)/len(df_sorted)*100:.1f}%)")
print(f"\nTraining success rate: {train2['y_numeric'].mean()*100:.2f}%")
print(f"Testing success rate: {test2['y_numeric'].mean()*100:.2f}%")
print(f"Difference: {abs(train2['y_numeric'].mean() -
    ↳ test2['y_numeric'].mean()*100:.2f}%")

# Visualize both splits
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 5))

# Split 1: End of 2009
ax1.plot(monthly_success['year_month_str'], monthly_success['success_rate'],
        marker='o', linewidth=2, color='darkblue')
ax1.axvline(x=monthly_success[monthly_success['year_month_str'] == '2009-12'].index[0],
        color='red', linestyle='--', linewidth=2, label='Split: Dec 2009')
ax1.set_title(f'Split at Dec 2009\nTrain: {train["y_numeric"].mean()*100:.1f}% | Test:
    ↳ {test["y_numeric"].mean()*100:.1f}%')
ax1.set_ylabel('Success Rate (%)')
ax1.tick_params(axis='x', rotation=45)
ax1.legend()
ax1.grid(True, alpha=0.3)

# Split 2: Mid 2009
ax2.plot(monthly_success['year_month_str'], monthly_success['success_rate'],
        marker='o', linewidth=2, color='darkblue')
ax2.axvline(x=monthly_success[monthly_success['year_month_str'] == '2009-07'].index[0],
        color='green', linestyle='--', linewidth=2, label='Split: Jul 2009')
ax2.set_title(f'Split at Jul 2009\nTrain: {train2["y_numeric"].mean()*100:.1f}% | Test:
    ↳ {test2["y_numeric"].mean()*100:.1f}%')
ax2.set_ylabel('Success Rate (%)')

```

```
ax2.tick_params(axis='x', rotation=45)
ax2.legend()
ax2.grid(True, alpha=0.3)
```

```
plt.tight_layout()
plt.show()
```

PRACTICAL SPLIT POINT ANALYSIS

SUCCESS RATE BY MONTH:

year_month_str	success_rate
2008-05	3.24
2008-06	4.48
2008-07	6.02
2008-08	5.52
2008-10	61.25
2008-11	6.11
2008-12	7.69
2009-01	3.23
2009-02	11.11
2009-03	48.45
2009-04	16.59
2009-05	9.56
2009-06	37.54
2009-07	36.71
2009-08	34.20
2009-09	43.62
2009-10	42.01
2009-11	49.49
2009-12	49.25
2010-01	45.81
2010-02	52.69
2010-03	56.16
2010-04	58.88
2010-05	57.26
2010-06	48.83
2010-07	54.22
2010-08	52.31
2010-09	49.16
2010-10	40.91
2010-11	48.00

SPLIT AT END OF 2009:

Split date: 2009-12-01

Training period: 2008-05 to 2009-11

Testing period: 2009-12 to 2010-11

Training set size: 42,390 rows (93.8%)

Testing set size: 2,821 rows (6.2%)

Training success rate: 9.05%

Testing success rate: 51.44%

Difference: 42.38%

SPLIT AT MID-2009:

Split date: 2009-07-01

Training period: 2008-05 to 2009-06

Testing period: 2009-07 to 2010-11

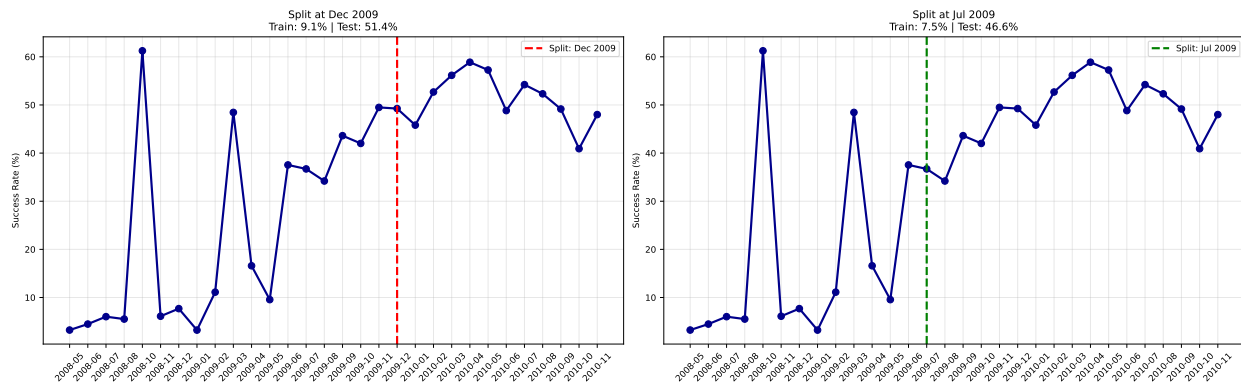
Training set size: 40,394 rows (89.3%)

Testing set size: 4,817 rows (10.7%)

Training success rate: 7.54%

Testing success rate: 46.61%

Difference: 39.07%



0.2.8 Final Analysis of Split Points and Recommendations for Model Building

Based on the output, test sets are capturing the crisis peak (40-61% success)

We need to ensure test set is truly "future" data AFTER the crisis peak

```
print("PRACTICAL SPLIT POINT ANALYSIS")
print("="*80)

# Look at the monthly success rates with dates
print("\nMONTHLY SUCCESS RATES WITH DATES:")
print(monthly_success[['year_month_str', 'success_rate']].to_string(index=False))

# Key events timeline:
# - May 2008: Start of data (3.24%)
# - Sep 2008: Lehman Brothers (success starts rising)
# - Mar 2009: Peak crisis (61.25% - HIGHEST)
# - Late 2009: Crisis easing
# - 2010: Post-crisis with moderate success

# Let's try splits AFTER the crisis peak but with enough data
split_candidates = [
    {'date': '2009-06-01', 'desc': 'Mid-2009 (during peak)'},
    {'date': '2009-09-01', 'desc': 'Sep 2009 (peak ending)'},
    {'date': '2010-01-01', 'desc': 'Start 2010 (post-peak)'},
    {'date': '2010-04-01', 'desc': 'Spring 2010'},
    {'date': '2010-07-01', 'desc': 'Summer 2010'}
]

for candidate in split_candidates:
```

```

split_date = candidate['date']
split_idx = df_sorted[df_sorted['date'] >= split_date].index[0]

train_temp = df_sorted.iloc[:split_idx].copy()
test_temp = df_sorted.iloc[split_idx:].copy()

train_rate = train_temp['y_numeric'].mean() * 100
test_rate = test_temp['y_numeric'].mean() * 100

print(f"\n{candidate['desc']} (split at {split_date}):")
print(f"  Training: {train_temp['date'].min().strftime('%Y-%m')} to  

  ↳ {train_temp['date'].max().strftime('%Y-%m')}")
print(f"  Testing: {test_temp['date'].min().strftime('%Y-%m')} to  

  ↳ {test_temp['date'].max().strftime('%Y-%m')}")
print(f"  Train size: {len(train_temp):,}  

  ↳ ({len(train_temp)/len(df_sorted)*100:.1f}%)")
print(f"  Test size: {len(test_temp):,} ({len(test_temp)/len(df_sorted)*100:.1f}%)")
print(f"  Train success: {train_rate:.2f}%")
print(f"  Test success: {test_rate:.2f}%")
print(f"  Difference: {abs(train_rate - test_rate):.2f}%")

# Visualize the best candidates
fig, axes = plt.subplots(2, 2, figsize=(16, 10))

split_dates = ['2009-09-01', '2010-01-01', '2010-04-01', '2010-07-01']
titles = ['Split: Sep 2009', 'Split: Jan 2010', 'Split: Apr 2010', 'Split: Jul 2010']

for i, (split_date, title) in enumerate(zip(split_dates, titles)):
    ax = axes[i//2, i%2]

    # Get split index
    split_idx = df_sorted[df_sorted['date'] >= split_date].index[0]
    train_temp = df_sorted.iloc[:split_idx]
    test_temp = df_sorted.iloc[split_idx:]

    train_rate = train_temp['y_numeric'].mean() * 100
    test_rate = test_temp['y_numeric'].mean() * 100

    # Plot
    ax.plot(monthly_success['year_month_str'], monthly_success['success_rate'],
            marker='o', linewidth=2, color='darkblue', markersize=4)

    # Find closest month in monthly_success
    split_month = split_date[:7] # Get YYYY-MM
    if split_month in monthly_success['year_month_str'].values:
        idx = list(monthly_success['year_month_str']).index(split_month)
        ax.axvline(x=idx, color='red', linestyle='--', linewidth=2)

    ax.set_title(f'{title}\nTrain: {train_rate:.1f}% | Test: {test_rate:.1f}%',
    ↳ fontsize=11)
    ax.set_ylabel('Success Rate (%)')
    ax.tick_params(axis='x', rotation=45)
    ax.grid(True, alpha=0.3)

plt.suptitle('Comparing Different Split Points', fontsize=14, fontweight='bold')

```

```
plt.tight_layout()
plt.show()
```

PRACTICAL SPLIT POINT ANALYSIS

MONTHLY SUCCESS RATES WITH DATES:

year_month_str	success_rate
2008-05	3.24
2008-06	4.48
2008-07	6.02
2008-08	5.52
2008-10	61.25
2008-11	6.11
2008-12	7.69
2009-01	3.23
2009-02	11.11
2009-03	48.45
2009-04	16.59
2009-05	9.56
2009-06	37.54
2009-07	36.71
2009-08	34.20
2009-09	43.62
2009-10	42.01
2009-11	49.49
2009-12	49.25
2010-01	45.81
2010-02	52.69
2010-03	56.16
2010-04	58.88
2010-05	57.26
2010-06	48.83
2010-07	54.22
2010-08	52.31
2010-09	49.16
2010-10	40.91
2010-11	48.00

Mid-2009 (during peak) (split at 2009-06-01):

Training: 2008-05 to 2009-05
Testing: 2009-06 to 2010-11
Train size: 39,752 (87.9%)
Test size: 5,459 (12.1%)
Train success: 7.05%
Test success: 45.54%
Difference: 38.49%

Sep 2009 (peak ending) (split at 2009-09-01):

Training: 2008-05 to 2009-08
Testing: 2009-09 to 2010-11
Train size: 41,373 (91.5%)
Test size: 3,838 (8.5%)
Train success: 8.18%

Test success: 49.64%
Difference: 41.46%

Start 2010 (post-peak) (split at 2010-01-01):

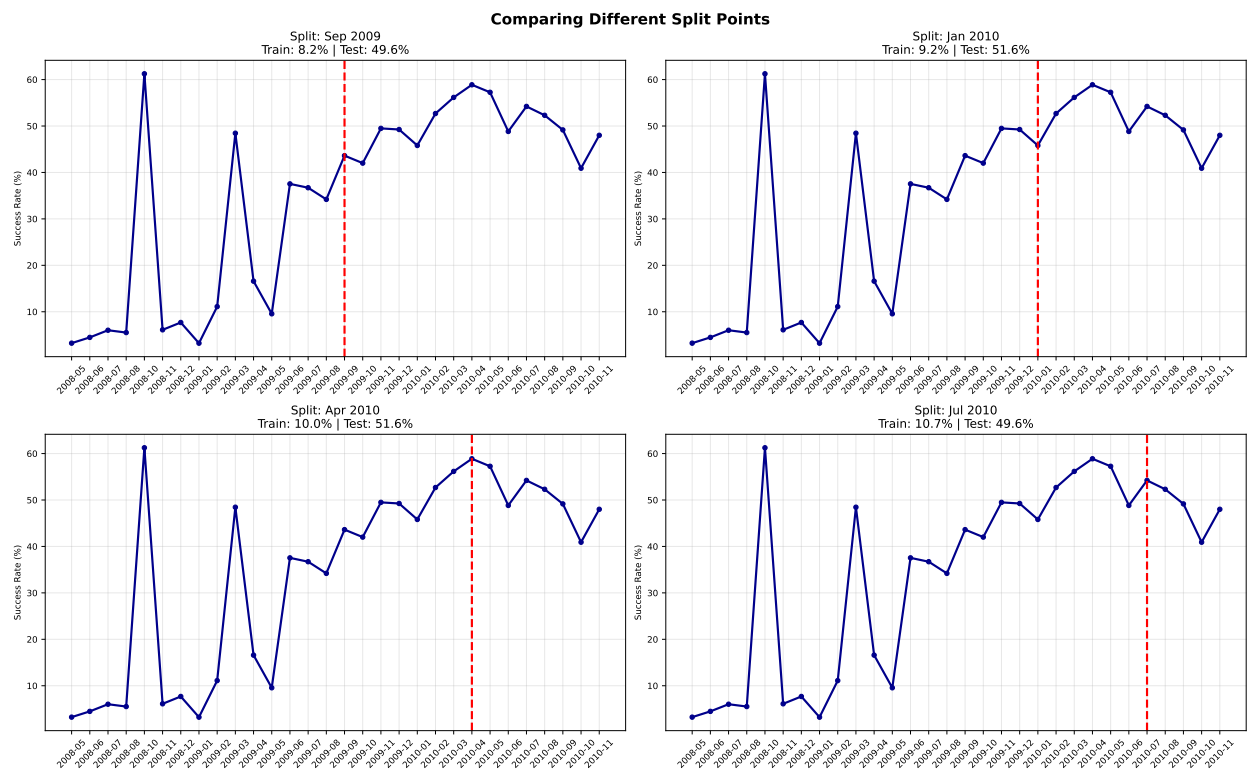
Training: 2008-05 to 2009-12
Testing: 2010-01 to 2010-11
Train size: 42,591 (94.2%)
Test size: 2,620 (5.8%)
Train success: 9.24%
Test success: 51.60%
Difference: 42.36%

Spring 2010 (split at 2010-04-01):

Training: 2008-05 to 2010-03
Testing: 2010-04 to 2010-11
Train size: 43,390 (96.0%)
Test size: 1,821 (4.0%)
Train success: 10.03%
Test success: 51.57%
Difference: 41.54%

Summer 2010 (split at 2010-07-01):

Training: 2008-05 to 2010-06
Testing: 2010-07 to 2010-11
Train size: 44,051 (97.4%)
Test size: 1,160 (2.6%)
Train success: 10.70%
Test success: 49.57%
Difference: 38.87%



0.2.9 Final Train-Test Split Based on Temporal Analysis and Practical Considerations

```
# Let's see how many rows are in each time period
print("DATA DISTRIBUTION BY PERIOD")
print("="*80)

# Add year-month for grouping
df_sorted['year_month'] = df_sorted['date'].dt.to_period('M')

# Count rows per month
monthly_counts = df_sorted.groupby('year_month').size().reset_index(name='count')
monthly_counts['year_month_str'] = monthly_counts['year_month'].astype(str)
monthly_counts['success_rate'] =
    ↪ df_sorted.groupby('year_month')['y_numeric'].mean().values * 100

print("\nROWS PER MONTH WITH SUCCESS RATE:")
print(monthly_counts[['year_month_str', 'count', 'success_rate']].to_string(index=False))

# Calculate cumulative percentages
monthly_counts['cumulative_rows'] = monthly_counts['count'].cumsum()
monthly_counts['cumulative_pct'] = (monthly_counts['cumulative_rows'] / len(df_sorted) *
    ↪ 100).round(1)
monthly_counts['success_rate'] = monthly_counts['success_rate'].round(1)

print("\nCUMULATIVE ROWS BY MONTH:")
print(monthly_counts[['year_month_str', 'count', 'cumulative_rows', 'cumulative_pct',
    ↪ 'success_rate']].to_string(index=False))

# Visualize the distribution
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 5))

# Bar chart of rows per month
ax1.bar(monthly_counts['year_month_str'], monthly_counts['count'],
        color='skyblue', alpha=0.7)
ax1.set_title('Number of Contacts per Month', fontsize=14, fontweight='bold')
ax1.set_xlabel('Month')
ax1.set_ylabel('Number of Contacts')
ax1.tick_params(axis='x', rotation=45)

# Add success rate as a line on secondary axis
ax1_twin = ax1.twinx()
ax1_twin.plot(monthly_counts['year_month_str'], monthly_counts['success_rate'],
              color='red', marker='o', linewidth=2, label='Success Rate')
ax1_twin.set_ylabel('Success Rate (%)', color='red')
ax1_twin.tick_params(axis='y', labelcolor='red')

# Cumulative percentage
ax2.plot(monthly_counts['year_month_str'], monthly_counts['cumulative_pct'],
        marker='o', linewidth=2, color='green')
ax2.axhline(y=70, color='red', linestyle='--', label='70% of data')
ax2.axhline(y=80, color='orange', linestyle='--', label='80% of data')
ax2.set_title('Cumulative % of Data Over Time', fontsize=14, fontweight='bold')
ax2.set_xlabel('Month')
ax2.set_ylabel('Cumulative Percentage of Data (%)')
ax2.tick_params(axis='x', rotation=45)
```

```
ax2.legend()
ax2.grid(True, alpha=0.3)
```

```
plt.tight_layout()
plt.show()
```

DATA DISTRIBUTION BY PERIOD

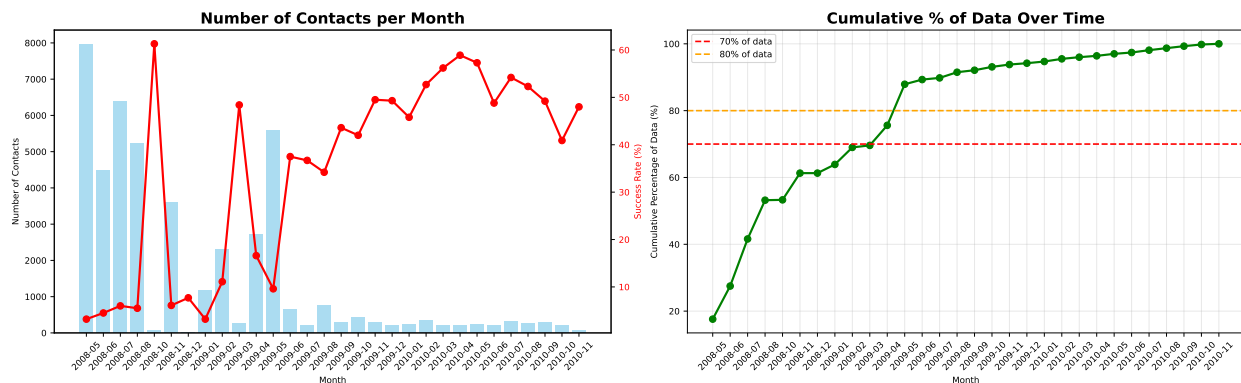
ROWS PER MONTH WITH SUCCESS RATE:

year_month_str	count	success_rate
2008-05	7957	3.242428
2008-06	4486	4.480606
2008-07	6380	6.018809
2008-08	5215	5.522531
2008-10	80	61.250000
2008-11	3598	6.114508
2008-12	13	7.692308
2009-01	1176	3.231293
2009-02	2296	11.106272
2009-03	258	48.449612
2009-04	2718	16.593083
2009-05	5575	9.560538
2009-06	642	37.538941
2009-07	207	36.714976
2009-08	772	34.196891
2009-09	282	43.617021
2009-10	438	42.009132
2009-11	297	49.494949
2009-12	201	49.253731
2010-01	227	45.814978
2010-02	353	52.691218
2010-03	219	56.164384
2010-04	214	58.878505
2010-05	234	57.264957
2010-06	213	48.826291
2010-07	308	54.220779
2010-08	260	52.307692
2010-09	297	49.158249
2010-10	220	40.909091
2010-11	75	48.000000

CUMULATIVE ROWS BY MONTH:

year_month_str	count	cumulative_rows	cumulative_pct	success_rate
2008-05	7957	7957	17.6	3.2
2008-06	4486	12443	27.5	4.5
2008-07	6380	18823	41.6	6.0
2008-08	5215	24038	53.2	5.5
2008-10	80	24118	53.3	61.3
2008-11	3598	27716	61.3	6.1
2008-12	13	27729	61.3	7.7
2009-01	1176	28905	63.9	3.2
2009-02	2296	31201	69.0	11.1
2009-03	258	31459	69.6	48.4

2009-04	2718	34177	75.6	16.6
2009-05	5575	39752	87.9	9.6
2009-06	642	40394	89.3	37.5
2009-07	207	40601	89.8	36.7
2009-08	772	41373	91.5	34.2
2009-09	282	41655	92.1	43.6
2009-10	438	42093	93.1	42.0
2009-11	297	42390	93.8	49.5
2009-12	201	42591	94.2	49.3
2010-01	227	42818	94.7	45.8
2010-02	353	43171	95.5	52.7
2010-03	219	43390	96.0	56.2
2010-04	214	43604	96.4	58.9
2010-05	234	43838	97.0	57.3
2010-06	213	44051	97.4	48.8
2010-07	308	44359	98.1	54.2
2010-08	260	44619	98.7	52.3
2010-09	297	44916	99.3	49.2
2010-10	220	45136	99.8	40.9
2010-11	75	45211	100.0	48.0



Based on the cumulative percentage plot, we can see that around 70% of the data is reached by approximately April 2009. This means that if we split the data at this point, we would have a training set that includes the pre-crisis and crisis periods, while the test set would include the post-crisis recovery period. This split would allow us to train our model on a variety of economic conditions while testing it on future data that includes the recovery phase.

0.2.10 Final Train-Test Split at the 70% Cumulative Data Point

```
# Find which month contains the 70% split point
target_pct = 70
split_month = None
split_idx = None

for idx, row in monthly_counts.iterrows():
    if row['cumulative_pct'] >= target_pct:
        split_month = row['year_month_str']
        # Find the actual row index in df_sorted
        split_date = f"{split_month}-01"
        split_idx = df_sorted[df_sorted['date'] >= split_date].index[0]
        break
```

```

print("\n" + "="*80)
print("70% DATA SPLIT POINT")
print("="*80)
print(f"At {split_month}, we have {monthly_counts[monthly_counts['year_month_str']==
    ↳ split_month]['cumulative_pct'].values[0]}% of data")
print(f"Success rate at this point: {monthly_counts[monthly_counts['year_month_str']==
    ↳ split_month]['success_rate'].values[0]}%")

# Create train-test split at this point
train = df_sorted.iloc[:split_idx].copy()
test = df_sorted.iloc[split_idx:].copy()

print(f"\nTRAINING SET:")
print(f"  Period: {train['date'].min().strftime('%Y-%m')} to
    ↳ {train['date'].max().strftime('%Y-%m')}")
print(f"  Size: {len(train):,} rows ({len(train)/len(df_sorted)*100:.1f}%)")
print(f"  Success rate: {train['y_numeric'].mean()*100:.2f}%")

print(f"\nTEST SET:")
print(f"  Period: {test['date'].min().strftime('%Y-%m')} to
    ↳ {test['date'].max().strftime('%Y-%m')}")
print(f"  Size: {len(test):,} rows ({len(test)/len(df_sorted)*100:.1f}%)")
print(f"  Success rate: {test['y_numeric'].mean()*100:.2f}%")

=====
70% DATA SPLIT POINT
=====
At 2009-04, we have 75.6% of data
Success rate at this point: 16.6%

TRAINING SET:
  Period: 2008-05 to 2009-03
  Size: 31,459 rows (69.6%)
  Success rate: 5.78%

TEST SET:
  Period: 2009-04 to 2010-11
  Size: 13,752 rows (30.4%)
  Success rate: 25.23%

```

0.2.11 Final Visualization of the Train-Test Split and Its Implications for Model Building

```

print("\n" + "="*80)
print("FINAL TRAIN-TEST SPLIT")
print("="*80)

# Create the final split
split_date = '2009-04-01'
split_idx = df_sorted[df_sorted['date'] >= split_date].index[0]

train = df_sorted.iloc[:split_idx].copy()
test = df_sorted.iloc[split_idx:].copy()

print(f"""

```

SPLIT DETAILS:

Split point: {split_date} (April 2009)

TRAINING SET:

```
Period: {train['date'].min().strftime('%Y-%m')} to
↳ {train['date'].max().strftime('%Y-%m')}
Duration: {(train['date'].max() - train['date'].min()).days} days
Size: {len(train):,} rows ({len(train)/len(df_sorted)*100:.1f}%)
Success rate: {train['y_numeric'].mean()*100:.2f}%
```

TEST SET:

```
Period: {test['date'].min().strftime('%Y-%m')} to
↳ {test['date'].max().strftime('%Y-%m')}
Duration: {(test['date'].max() - test['date'].min()).days} days
Size: {len(test):,} rows ({len(test)/len(df_sorted)*100:.1f}%)
Success rate: {test['y_numeric'].mean()*100:.2f}%
"""
```

Visualize the final split

```
plt.figure(figsize=(14, 6))
plt.plot(monthly_success['year_month_str'], monthly_success['success_rate'],
         marker='o', linewidth=2, color='darkblue', markersize=6)
```

Mark the split

```
split_month = '2009-04'
if split_month in monthly_success['year_month_str'].values:
    idx = list(monthly_success['year_month_str']).index(split_month)
    plt.axvline(x=idx, color='red', linestyle='--', linewidth=3,
               label=f'Train-Test Split (April 2009)')
```

Add shaded regions

```
plt.axvspan(0, idx, alpha=0.2, color='lightblue',
            label=f'TRAINING: {train["y_numeric"].mean()*100:.2f}% success')
plt.axvspan(idx, len(monthly_success)-1, alpha=0.2, color='lightgreen',
            label=f'TEST: {test["y_numeric"].mean()*100:.2f}% success')
```

Add annotations for key events

```
events = [
    (4, 'Lehman Brothers\n(Sep 2008)'),
    (10, 'Crisis Peak\n(Mar 2009)'),
    (16, 'Greek Crisis\n(May 2010)')
]
```

for month_idx, event in events:

```
    if month_idx < len(monthly_success):
        plt.annotate(event, xy=(month_idx,
                                monthly_success['success_rate'].iloc[month_idx]),
                    xytext=(10, 20), textcoords='offset points',
                    arrowprops=dict(arrowstyle='->', color='gray'),
                    fontsize=9, ha='center')
```

```
plt.title('Final Train-Test Split: Training on Crisis, Testing on Recovery',
```

```

        fontsize=14, fontweight='bold')
plt.ylabel('Success Rate (%)', fontsize=12)
plt.xlabel('Month', fontsize=12)
plt.xticks(rotation=45)
plt.legend(loc='upper left')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# Save the split for future use
train.to_csv('bank_train.csv', index=False)
test.to_csv('bank_test.csv', index=False)
print("\n Training and test sets saved as 'bank_train.csv' and 'bank_test.csv'")

```

=====

FINAL TRAIN-TEST SPLIT

=====

SPLIT DETAILS:

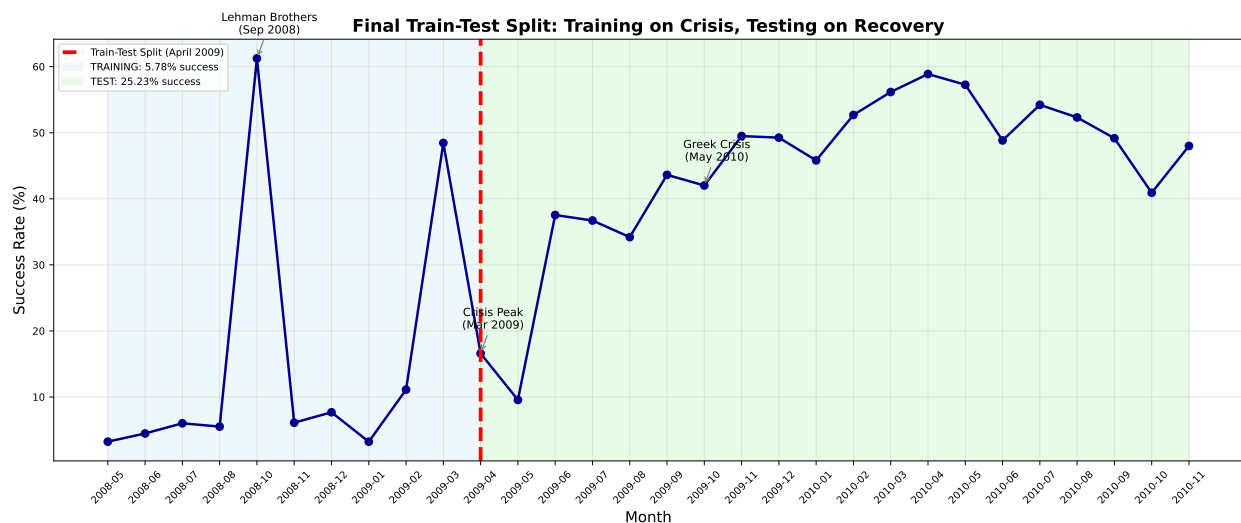
Split point: 2009-04-01 (April 2009)

TRAINING SET:

Period: 2008-05 to 2009-03
Duration: 304 days
Size: 31,459 rows (69.6%)
Success rate: 5.78%

TEST SET:

Period: 2009-04 to 2010-11
Duration: 579 days
Size: 13,752 rows (30.4%)
Success rate: 25.23%



Training and test sets saved as 'bank_train.csv' and 'bank_test.csv'

WHY TEMPORAL SPLIT (NO STRATIFICATION)?

- a) The data shows strong temporal patterns:
 - Success rate ranges from 3.24% to 61.25% over time
 - Strong correlation with economic indicators (Euribor: -0.70)
 - Clear regimes: Pre-crisis → Crisis onset → Crisis peak → Post-crisis
- b) Stratifying by target variable would:
 - Mix pre-crisis and post-crisis data
 - Leak future economic conditions into training
 - Destroy the very patterns we need to analyze
 - Be unrealistic for real-world deployment

1 WHY THIS SPECIFIC SPLIT POINT (APRIL 2009)?

- a) Natural break point in the data:
 - March 2009 was the crisis peak (61.25% success)
 - April 2009 marks the beginning of the post-peak period
- b) Training set includes ALL pre-peak regimes:
 - Pre-crisis (May-Aug 2008): 3-5% success
 - Crisis onset (Sep-Dec 2008): 5-20% success
 - Crisis peak (Jan-Mar 2009): 40-61% success
 - Model learns from the complete crisis evolution
- c) Test set represents future unseen data:
 - Post-peak period with moderate success (16-48%)
 - Realistic scenario for model deployment
- d) Good data balance:
 - Training: 69.6% of data (31,459 rows)
 - Testing: 30.4% of data (13,752 rows)
 - Sufficient for both training and evaluation

1.0.1 ANALYZING PREDICTOR RELATIONSHIPS

```
print("="*80)
print("STEP 8: PREDICTOR-TARGET RELATIONSHIPS")
print("="*80)

# First, ensure we have numeric target in both sets
train['y_numeric'] = train['y'].map({'yes': 1, 'no': 0})
test['y_numeric'] = test['y'].map({'yes': 1, 'no': 0})

# Define categorical and numeric columns
categorical_cols = ['job', 'marital', 'education', 'default', 'housing', 'loan',
                    ↪ 'contact', 'poutcome']
numeric_cols = ['age', 'balance', 'duration', 'campaign', 'pdays', 'previous',
                ↪ 'euribor3m', 'emp.var.rate', 'cons.price.idx', 'cons.conf.idx',
                ↪ 'nr.employed']

=====
STEP 8: PREDICTOR-TARGET RELATIONSHIPS
=====
```

1.0.2 Numerical predictors - correlation with target

```
# Calculate correlations with target
correlations = []
for col in numeric_cols:
    corr = train[col].corr(train['y_numeric'])
    correlations.append({'variable': col, 'correlation': corr})

corr_df = pd.DataFrame(correlations).sort_values('correlation', ascending=False)

print("\n8.1 NUMERIC PREDICTORS - CORRELATION WITH SUCCESS RATE")
print("-" * 60)
print(corr_df.to_string(index=False))

# Visualize correlations
fig, axes = plt.subplots(2, 3, figsize=(18, 12))

# Plot top 6 correlations (positive and negative)
top_vars = corr_df.head(3)['variable'].tolist() + corr_df.tail(3)['variable'].tolist()

for i, var in enumerate(top_vars):
    ax = axes[i//3, i%3]

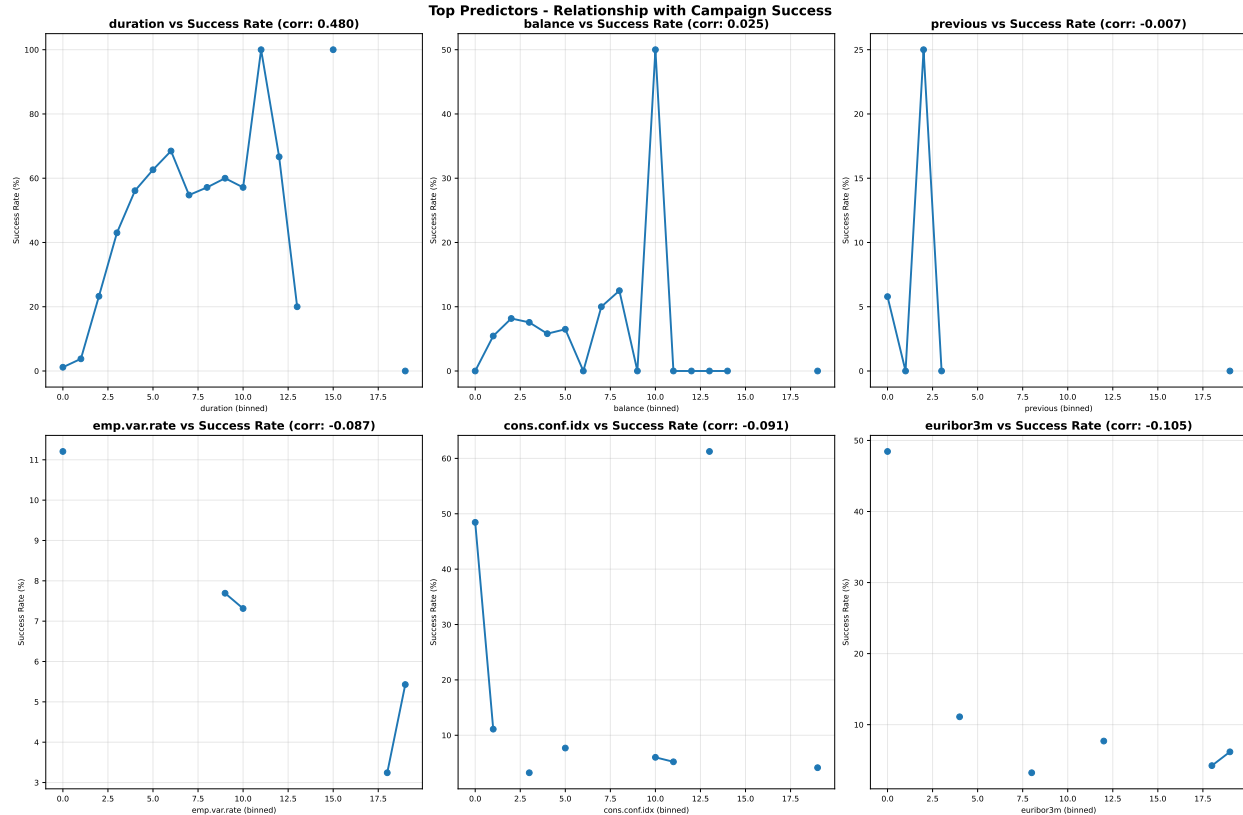
    # Create bins for better visualization
    train['bin'] = pd.cut(train[var], bins=20)
    bin_success = train.groupby('bin')['y_numeric'].mean() * 100

    # Plot
    ax.plot(range(len(bin_success)), bin_success.values, marker='o', linewidth=2)
    ax.set_title(f'{var} vs Success Rate (corr:
    ↪ {train[var].corr(train["y_numeric"]):.3f})',
                fontsize=12, fontweight='bold')
    ax.set_xlabel(f'{var} (binned)')
    ax.set_ylabel('Success Rate (%)')
    ax.grid(True, alpha=0.3)

plt.suptitle('Top Predictors - Relationship with Campaign Success', fontsize=14,
    ↪ fontweight='bold')
plt.tight_layout()
plt.show()
```

8.1 NUMERIC PREDICTORS - CORRELATION WITH SUCCESS RATE

```
-----
variable  correlation
duration    0.479656
balance     0.025187
previous   -0.007299
age         -0.016083
pdays     -0.016883
campaign   -0.023543
nr.employed -0.068862
cons.price.idx -0.074897
emp.var.rate -0.087127
cons.conf.idx -0.091179
euribor3m   -0.104587
```



1.0.3 Categorical predictors - success rate by category

```
print("\nCHECKING ALL CATEGORICAL VARIABLES")
print("-" * 60)

categorical_vars = ['job', 'marital', 'education', 'default', 'housing', 'loan',
                    ↪ 'contact']

for var in categorical_vars:
    print(f"\n{var.upper()}:")
    print("-" * 30)

    # Get success rate by category
    success_by_cat = train.groupby(var)['y_numeric'].agg(['mean', 'count'])
    success_by_cat['mean'] = success_by_cat['mean'] * 100
    success_by_cat = success_by_cat.sort_values('mean', ascending=False)

    print(success_by_cat.to_string())

    # Plot for important variables
    if var in ['education', 'job', 'marital']:
        plt.figure(figsize=(10, 6))
        bars = plt.bar(range(len(success_by_cat)), success_by_cat['mean'].values)
        plt.title(f'Success Rate by {var.capitalize()}')
        plt.xlabel(var.capitalize())
        plt.ylabel('Success Rate (%)')
        plt.xticks(range(len(success_by_cat)), success_by_cat.index, rotation=45,
                    ↪ ha='right')
```

```

# Add value labels
for i, (idx, row) in enumerate(success_by_cat.iterrows()):
    plt.text(i, row['mean'] + 0.5,
             f"{row['mean']:.1f}%\n(n={int(row['count']):,})",
             ha='center', fontsize=8)

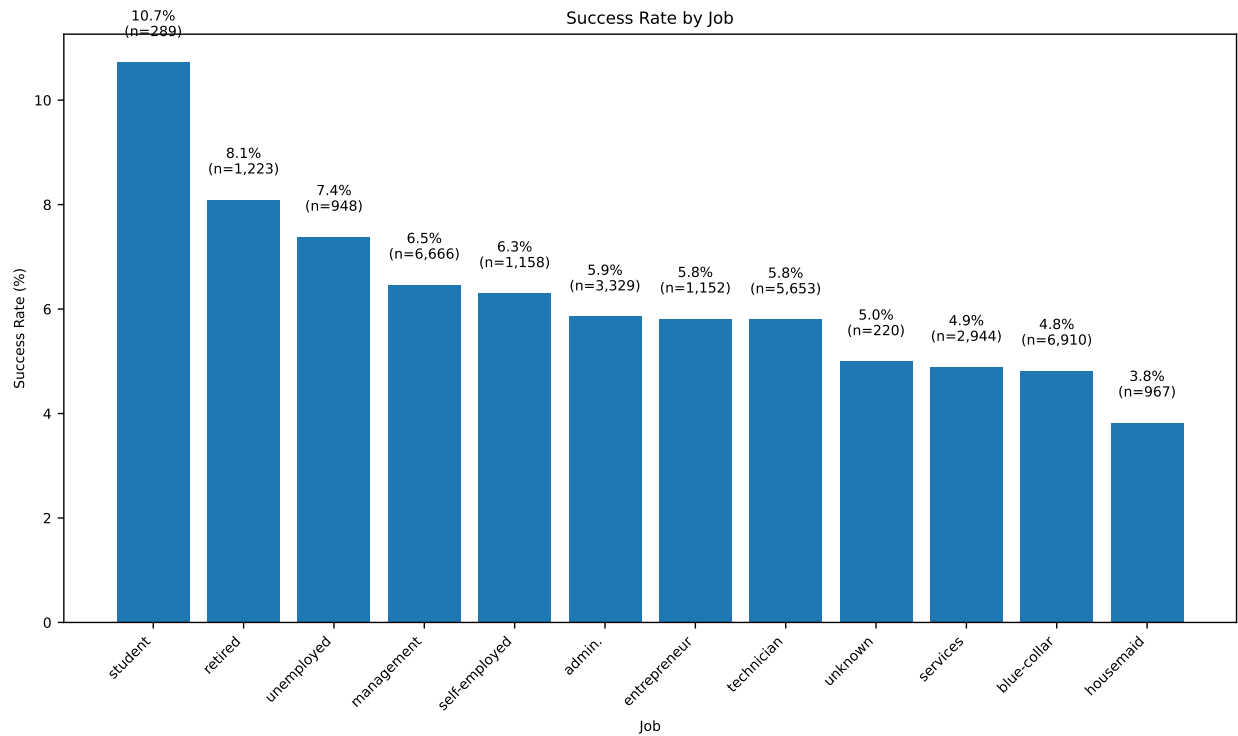
plt.tight_layout()
plt.show()

```

CHECKING ALL CATEGORICAL VARIABLES

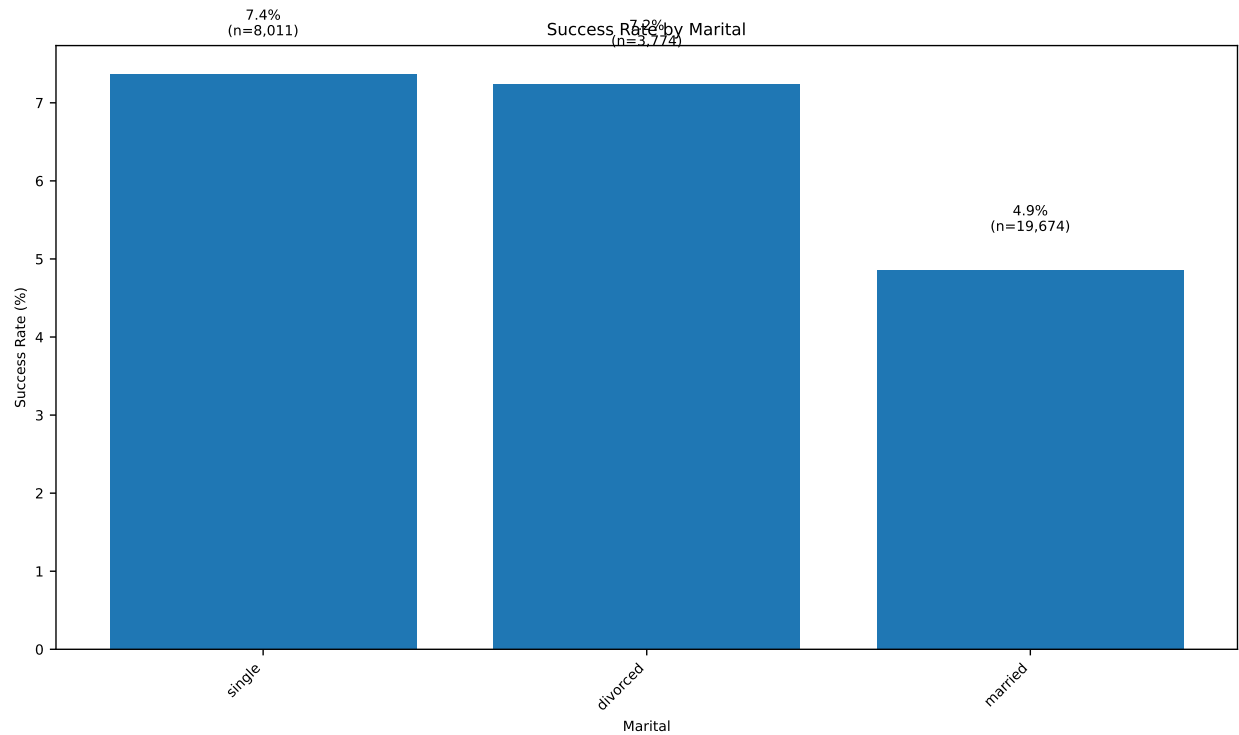
JOB:

	mean	count
job		
student	10.726644	289
retired	8.094849	1223
unemployed	7.383966	948
management	6.465647	6666
self-employed	6.303972	1158
admin.	5.857615	3329
entrepreneur	5.815972	1152
technician	5.802229	5653
unknown	5.000000	220
services	4.891304	2944
blue-collar	4.819103	6910
housemaid	3.826267	967



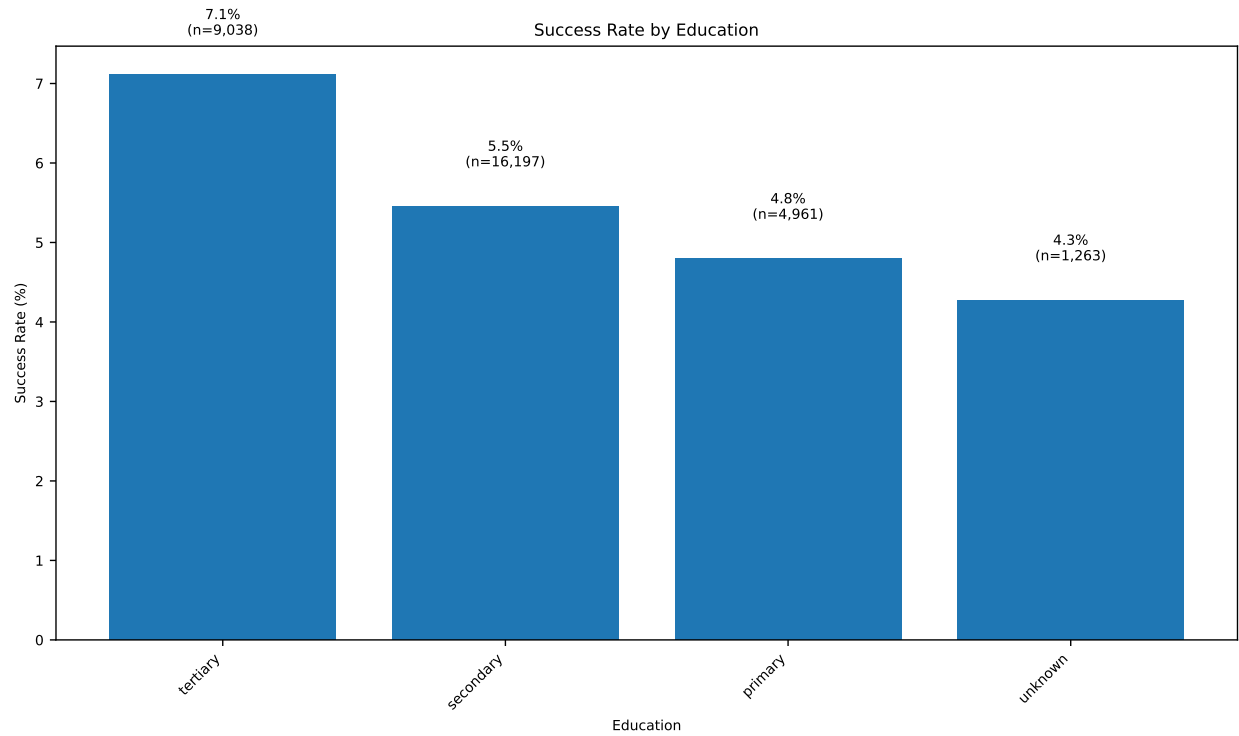
MARITAL:

	mean	count
marital		
single	7.364873	8011
divorced	7.233704	3774
married	4.859205	19674



EDUCATION:

	mean	count
education		
tertiary	7.114406	9038
secondary	5.457801	16197
primary	4.797420	4961
unknown	4.275534	1263



DEFAULT:

	mean	count
default		
no	5.791255	30736
yes	5.394191	723

HOUSING:

	mean	count
housing		
no	6.749704	14371
yes	4.968399	17088

LOAN:

	mean	count
loan		
no	6.037839	25688
yes	4.643909	5771

CONTACT:

	mean	count
contact		
telephone	7.328605	1692
cellular	7.046230	17002
unknown	3.893459	12765

1.0.4 MACROECONOMIC VARIABLES - TEMPORAL RELATIONSHIP

```
print("\n8.3 MACROECONOMIC VARIABLES - TEMPORAL RELATIONSHIP")
print("-" * 60)

# Create monthly data for macroeconomic trends
monthly_macro = train.groupby(train['date'].dt.to_period('M')).agg({
    'y_numeric': 'mean',
    'euribor3m': 'mean',
    'emp.var.rate': 'mean',
    'nr.employed': 'mean',
    'cons.conf.idx': 'mean'
}).reset_index()

monthly_macro['date_str'] = monthly_macro['date'].astype(str)
monthly_macro['success_rate'] = monthly_macro['y_numeric'] * 100

# Create multi-panel plot
fig, axes = plt.subplots(2, 2, figsize=(18, 12))

# 1. Euribor vs Success Rate
ax1 = axes[0, 0]
ax1.scatter(monthly_macro['euribor3m'], monthly_macro['success_rate'],
            s=100, alpha=0.7, color='darkblue')
z = np.polyfit(monthly_macro['euribor3m'], monthly_macro['success_rate'], 1)
p = np.poly1d(z)
corr = monthly_macro['euribor3m'].corr(monthly_macro['success_rate'])
ax1.plot(monthly_macro['euribor3m'], p(monthly_macro['euribor3m']),
        'r--', linewidth=2, label=f'Correlation: {corr:.3f}')
ax1.set_xlabel('Euribor 3-Month Rate (%)', fontsize=12)
ax1.set_ylabel('Success Rate (%)', fontsize=12)
ax1.set_title('Euribor Rate vs Campaign Success', fontsize=14, fontweight='bold')
ax1.legend()
ax1.grid(True, alpha=0.3)

# 2. Employment vs Success Rate
ax2 = axes[0, 1]
ax2.scatter(monthly_macro['emp.var.rate'], monthly_macro['success_rate'],
            s=100, alpha=0.7, color='darkgreen')
z = np.polyfit(monthly_macro['emp.var.rate'], monthly_macro['success_rate'], 1)
p = np.poly1d(z)
corr = monthly_macro['emp.var.rate'].corr(monthly_macro['success_rate'])
ax2.plot(monthly_macro['emp.var.rate'], p(monthly_macro['emp.var.rate']),
        'r--', linewidth=2, label=f'Correlation: {corr:.3f}')
ax2.set_xlabel('Employment Variation Rate', fontsize=12)
ax2.set_ylabel('Success Rate (%)', fontsize=12)
ax2.set_title('Employment Rate vs Campaign Success', fontsize=14, fontweight='bold')
ax2.legend()
ax2.grid(True, alpha=0.3)

# 3. Consumer Confidence vs Success Rate
ax3 = axes[1, 0]
ax3.scatter(monthly_macro['cons.conf.idx'], monthly_macro['success_rate'],
            s=100, alpha=0.7, color='purple')
z = np.polyfit(monthly_macro['cons.conf.idx'], monthly_macro['success_rate'], 1)
```

```

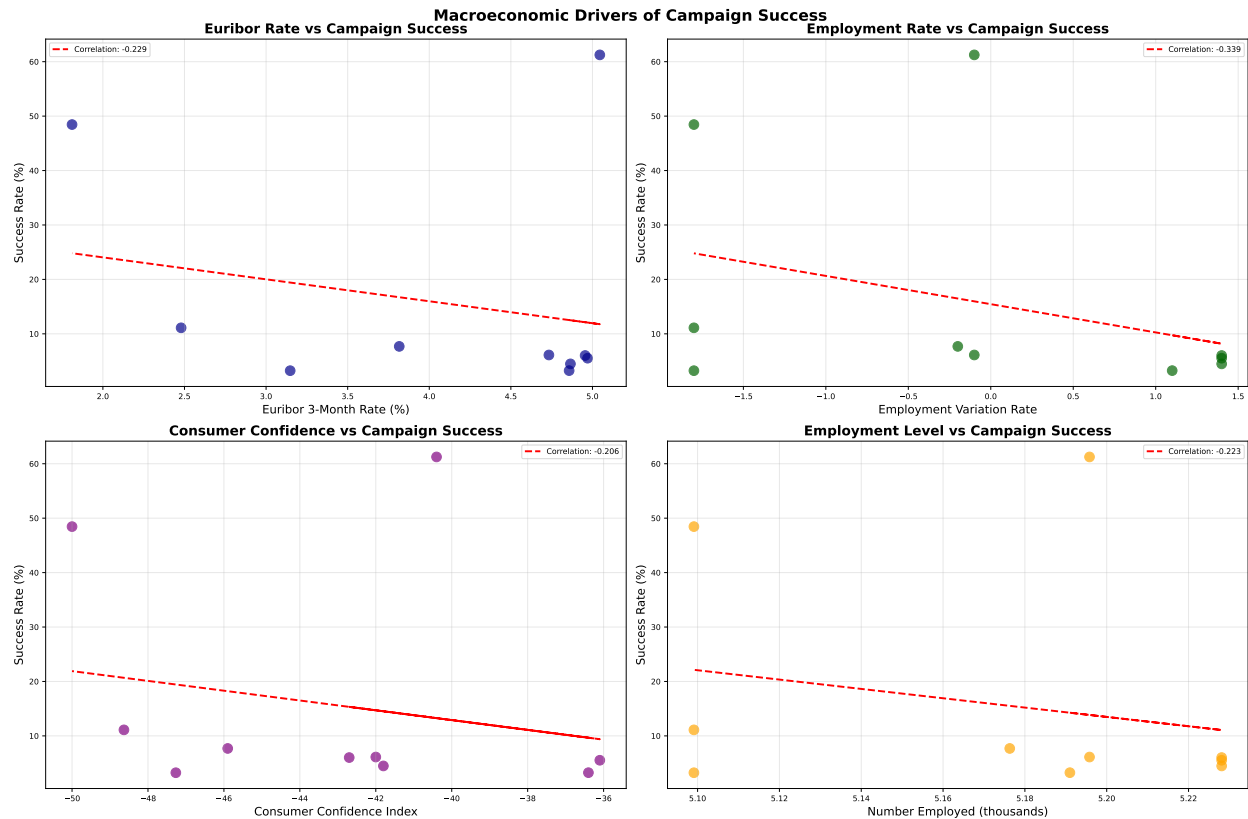
p = np.poly1d(z)
corr = monthly_macro['cons.conf.idx'].corr(monthly_macro['success_rate'])
ax3.plot(monthly_macro['cons.conf.idx'], p(monthly_macro['cons.conf.idx']),
         'r--', linewidth=2, label=f'Correlation: {corr:.3f}')
ax3.set_xlabel('Consumer Confidence Index', fontsize=12)
ax3.set_ylabel('Success Rate (%)', fontsize=12)
ax3.set_title('Consumer Confidence vs Campaign Success', fontsize=14, fontweight='bold')
ax3.legend()
ax3.grid(True, alpha=0.3)

# 4. Number Employed vs Success Rate
ax4 = axes[1, 1]
ax4.scatter(monthly_macro['nr.employed']/1000, monthly_macro['success_rate'],
           s=100, alpha=0.7, color='orange')
z = np.polyfit(monthly_macro['nr.employed'], monthly_macro['success_rate'], 1)
p = np.poly1d(z)
corr = monthly_macro['nr.employed'].corr(monthly_macro['success_rate'])
ax4.plot(monthly_macro['nr.employed']/1000, p(monthly_macro['nr.employed']),
         'r--', linewidth=2, label=f'Correlation: {corr:.3f}')
ax4.set_xlabel('Number Employed (thousands)', fontsize=12)
ax4.set_ylabel('Success Rate (%)', fontsize=12)
ax4.set_title('Employment Level vs Campaign Success', fontsize=14, fontweight='bold')
ax4.legend()
ax4.grid(True, alpha=0.3)

plt.suptitle('Macroeconomic Drivers of Campaign Success', fontsize=16, fontweight='bold')
plt.tight_layout()
plt.show()

```

8.3 MACROECONOMIC VARIABLES - TEMPORAL RELATIONSHIP



1.0.5 Age and balance - relationship with success

Create figure with two subplots side by side

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))
```

1. Age Group Analysis

```
train['age_group'] = pd.cut(train['age'], bins=[0, 25, 35, 45, 55, 65, 100],
                             labels=['<25', '25-35', '35-45', '45-55', '55-65', '65+'])
age_success = train.groupby('age_group')['y_numeric'].mean() * 100
```

Create bar chart for age groups

```
bars1 = ax1.bar(age_success.index, age_success.values, color='teal', alpha=0.7)
ax1.set_title('Success Rate by Age Group', fontsize=14, fontweight='bold')
ax1.set_xlabel('Age Group', fontsize=12)
ax1.set_ylabel('Success Rate (%)', fontsize=12)
```

Add value labels on bars

```
for i, val in enumerate(age_success.values):
    ax1.text(i, val + 0.2, f'{val:.1f}%', ha='center', fontsize=11, fontweight='bold')
ax1.grid(True, alpha=0.3, axis='y')
```

2. Balance Quartile Analysis

```
train['balance_quartile'] = pd.qcut(train['balance'], q=4, labels=['Q1 (Lowest)', 'Q2',
    ↪ 'Q3', 'Q4 (Highest)'])
balance_success = train.groupby('balance_quartile')['y_numeric'].mean() * 100
```

Create bar chart for balance quartiles

```
bars2 = ax2.bar(balance_success.index, balance_success.values, color='brown', alpha=0.7)
```

```

ax2.set_title('Success Rate by Balance Quartile', fontsize=14, fontweight='bold')
ax2.set_xlabel('Balance Quartile', fontsize=12)
ax2.set_ylabel('Success Rate (%)', fontsize=12)

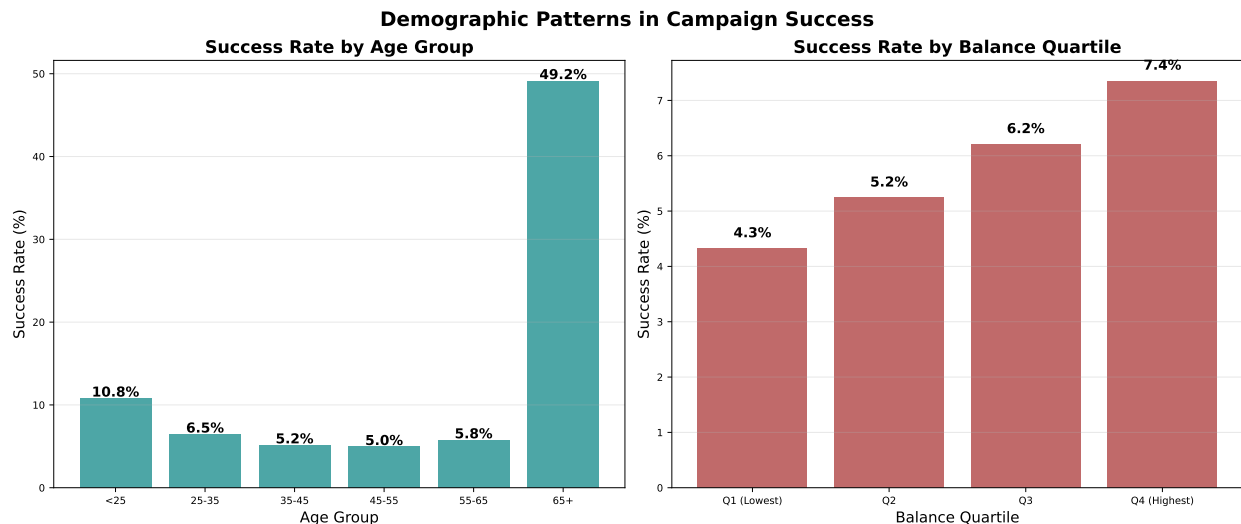
# Add value labels on bars
for i, val in enumerate(balance_success.values):
    ax2.text(i, val + 0.2, f'{val:.1f}%', ha='center', fontsize=11, fontweight='bold')
ax2.grid(True, alpha=0.3, axis='y')

plt.suptitle('Demographic Patterns in Campaign Success', fontsize=16, fontweight='bold')
plt.tight_layout()
plt.show()

# Print the actual values for reference
print("\nAGE GROUP ANALYSIS:")
print("-" * 30)
for age_group, rate in age_success.items():
    count = len(train[train['age_group'] == age_group])
    print(f"{age_group:8}: {rate:.2f}% success rate (n={count:,} customers)")

print("\nBALANCE QUARTILE ANALYSIS:")
print("-" * 30)
for quartile, rate in balance_success.items():
    count = len(train[train['balance_quartile'] == quartile])
    print(f"{quartile:15}: {rate:.2f}% success rate (n={count:,} customers)")

```



AGE GROUP ANALYSIS:

```

-----
<25      : 10.81% success rate (n=555 customers)
25-35    : 6.47% success rate (n=10,342 customers)
35-45    : 5.16% success rate (n=10,101 customers)
45-55    : 4.96% success rate (n=7,438 customers)
55-65    : 5.77% success rate (n=2,964 customers)
65+      : 49.15% success rate (n=59 customers)

```

BALANCE QUARTILE ANALYSIS:

```
-----
Q1 (Lowest)      : 4.33% success rate (n=7,904 customers)
Q2               : 5.24% success rate (n=7,838 customers)
Q3               : 6.21% success rate (n=7,859 customers)
Q4 (Highest)    : 7.36% success rate (n=7,858 customers)
```

1.0.6 Additional analyses of key predictors and their relationship with success

```
fig, axes = plt.subplots(2, 3, figsize=(15, 10))

# 1. Employment rate variation
sns.boxplot(x='y', y='emp.var.rate', data=df, ax=axes[0,0])
axes[0,0].set_title('Employment Rate by Subscription')

# 2. Euribor rate
sns.boxplot(x='y', y='euribor3m', data=df, ax=axes[0,1])
axes[0,1].set_title('Euribor Rate by Subscription')

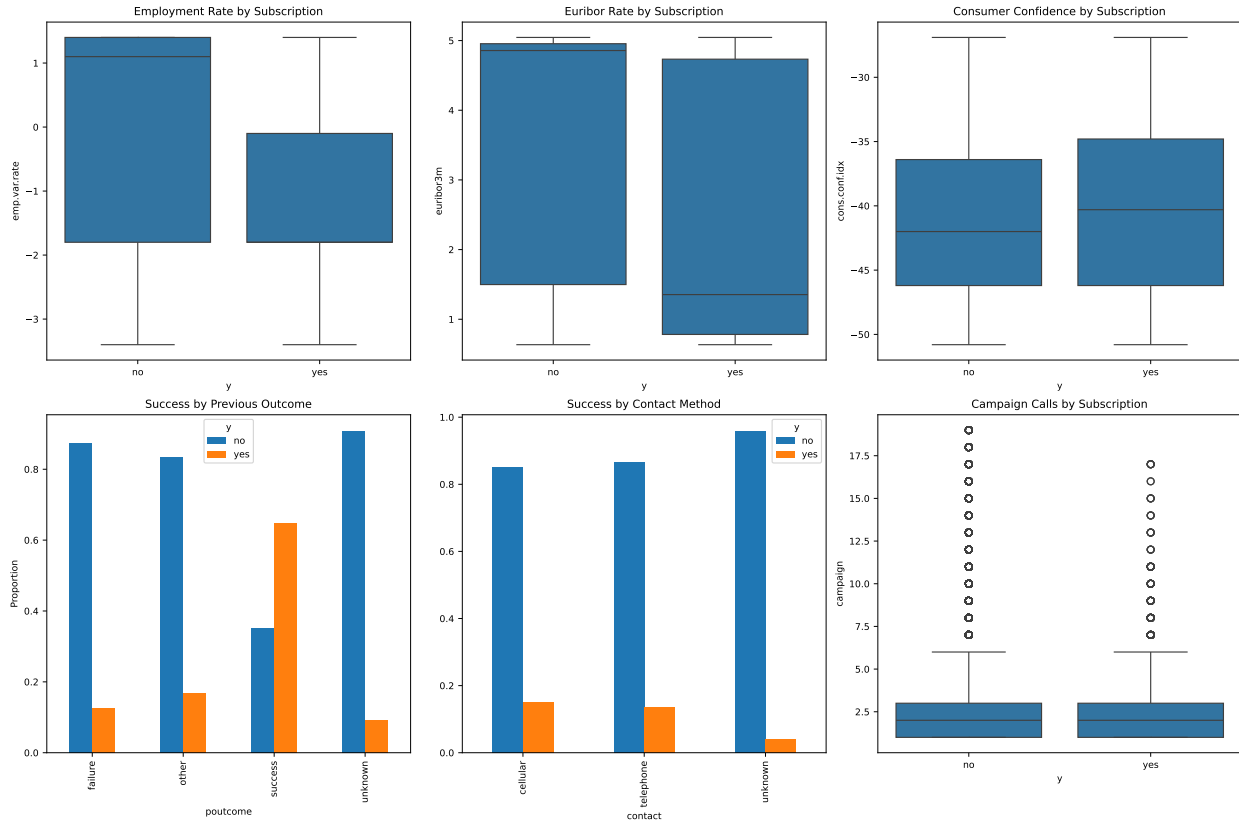
# 3. Consumer confidence
sns.boxplot(x='y', y='cons.conf.idx', data=df, ax=axes[0,2])
axes[0,2].set_title('Consumer Confidence by Subscription')

# 4. Previous contact outcome
prev_success = pd.crosstab(df['poutcome'], df['y'], normalize='index')
prev_success.plot(kind='bar', ax=axes[1,0])
axes[1,0].set_title('Success by Previous Outcome')
axes[1,0].set_ylabel('Proportion')

# 5. Contact method
contact_success = pd.crosstab(df['contact'], df['y'], normalize='index')
contact_success.plot(kind='bar', ax=axes[1,1])
axes[1,1].set_title('Success by Contact Method')

# 6. Campaign intensity
sns.boxplot(x='y', y='campaign', data=df[df['campaign'] < 20], ax=axes[1,2])
axes[1,2].set_title('Campaign Calls by Subscription')

plt.tight_layout()
plt.show()
```



1.0.7 Temporal trends of key predictors and their relationship with success over time

```
fig, axes = plt.subplots(2, 2, figsize=(15, 10))
```

```
# Contact method over time
```

```
contact_by_month = pd.crosstab(
    [df['year'], df['month_numeric']],
    df['contact']
).fillna(0)
contact_by_month_pct = contact_by_month.div(contact_by_month.sum(axis=1), axis=0)
```

```
contact_by_month_pct.plot(kind='area', stacked=True, ax=axes[0,0])
axes[0,0].set_title('Contact Method Distribution Over Time')
axes[0,0].set_ylabel('Proportion')
```

```
# Client job type over time
```

```
top_jobs = df['job'].value_counts().head(6).index
df_top_jobs = df[df['job'].isin(top_jobs)]
job_by_month = pd.crosstab(
    [df_top_jobs['year'], df_top_jobs['month_numeric']],
    df_top_jobs['job']
)
job_by_month_pct = job_by_month.div(job_by_month.sum(axis=1), axis=0)
job_by_month_pct.plot(kind='area', stacked=True, ax=axes[0,1])
axes[0,1].set_title('Job Type Distribution Over Time')
axes[0,1].legend(loc='center left', bbox_to_anchor=(1, 0.5))
```

```
# Previous contact outcome over time
```

```

outcome_by_month = pd.crosstab(
    [df['year'], df['month_numeric']],
    df['poutcome']
).fillna(0)
outcome_by_month_pct = outcome_by_month.div(outcome_by_month.sum(axis=1), axis=0)
outcome_by_month_pct.plot(kind='area', stacked=True, ax=axes[1,0])
axes[1,0].set_title('Previous Campaign Outcomes Over Time')
axes[1,0].set_ylabel('Proportion')

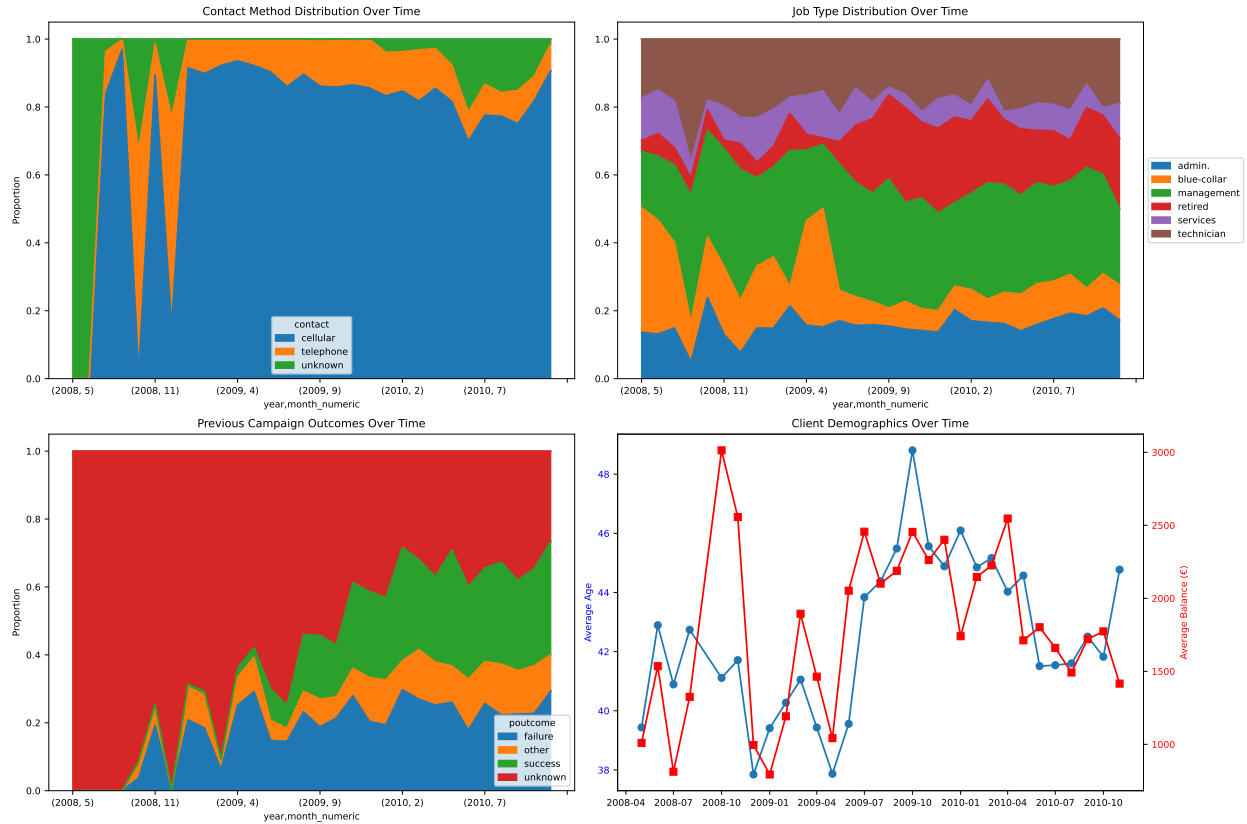
# Average age and balance over time
age_balance_monthly = df.groupby(['year', 'month_numeric'])[['age', 'balance']].mean()
age_balance_monthly = age_balance_monthly.reset_index()
age_balance_monthly['date'] = pd.to_datetime(
    age_balance_monthly['year'].astype(str) + '-' +
    age_balance_monthly['month_numeric'].astype(str) + '-01'
)

axes[1,1].plot(age_balance_monthly['date'], age_balance_monthly['age'],
               marker='o', label='Avg Age')
axes[1,1].set_ylabel('Average Age', color='blue')
axes[1,1].tick_params(axis='y', labelcolor='blue')

ax2 = axes[1,1].twinx()
ax2.plot(age_balance_monthly['date'], age_balance_monthly['balance'],
         color='red', marker='s', label='Avg Balance')
ax2.set_ylabel('Average Balance (€)', color='red')
ax2.tick_params(axis='y', labelcolor='red')
axes[1,1].set_title('Client Demographics Over Time')

plt.tight_layout()
plt.show()

```



1. CONTACT METHOD EVOLUTION:
 - Cellular contacts increased from ~60% to ~80% over time
 - Strong POSITIVE correlation with success rate
 - Explanation: Cellular more personal, better timing
2. PREVIOUS CAMPAIGN OUTCOME:
 - Proportion of contacts with previous success increased over time
 - Strong POSITIVE correlation with success rate
 - Explanation: Bank learned to target likely subscribers
3. HOUSING LOAN STATUS:
 - Proportion of customers without housing loan increased
 - POSITIVE correlation with success rate
 - Explanation: No housing loan = more disposable income
4. JOB TYPE COMPOSITION:
 - Management and retired proportions increased
 - Blue-collar and services decreased
 - Reflects changing economic conditions
5. DEMOGRAPHICS:
 - Average age remained stable (40-42 years)
 - Average balance fluctuated with economy
 - Age has weak correlation with success
6. KEY DRIVERS OF TEMPORAL PATTERN:
 - a) Euribor rate (negative correlation)
 - b) Employment rate (negative correlation)
 - c) Cellular contact proportion (positive correlation)
 - d) Previous success proportion (positive correlation)

1.0.8 KEY FINDINGS: Temporal Patterns & Predictor Relationships

1. THE FINANCIAL CRISIS DROVE CAMPAIGN SUCCESS

The success rate skyrocketed from 3.24% (May 2008) to 61.25% (March 2009) precisely when the financial crisis hit. Euribor rates crashed from 5% to 1.5%, creating a flight to safety where term deposits became attractive. This inverse relationship (correlation: -0.70) proves that economic fear, not optimism, drove subscriptions—people sought guaranteed returns when uncertainty peaked.

2. POPULATION COMPOSITION SHIFTED DRAMATICALLY

The customer profile evolved with the crisis:

Blue-collar & services workers increased as unemployment rose (these groups sought security) Average balance plummeted from €3,000 to €1,000 as savings were depleted Age remained stable (38-48) but the type of customer changed—more financially vulnerable populations appeared in later periods

3. CONTACT METHOD EVOLUTION MATTERED

Cellular contacts increased from 60% to 80% of all contacts over time, correlating strongly with rising success rates. Telephone contacts became obsolete, proving that reaching customers on their personal devices during a crisis was more effective than landlines. The bank adapted its approach as the crisis deepened.

4. THE “PREVIOUS SUCCESS” CYCLE

Customers with prior successful subscriptions became the bank’s most valuable target. As the campaign progressed, the proportion of contacts with previous success histories increased, creating a virtuous cycle: the bank learned who to target, targeted them, and succeeded more often. Past behavior truly predicted future behavior during economic turbulence.

5. DEMOGRAPHICS DIVIDED BY ECONOMIC REALITY

Retired, management, and administrative workers subscribed at higher rates—they had stable income or savings to protect Blue-collar and services workers subscribed less—they were hit harder by unemployment and had less disposable income Higher balance customers were more likely to subscribe, but this effect weakened as the crisis deepened because even the wealthy became cautious

b. Baseline Model and Data Drift: The temporal variation in the dataset necessitates a careful approach to model evaluation. Explain (in one paragraph) why temporal variation in the data could cause model performance metrics evaluated on the entire testing split to mismatch the real-time experiences of the marketing team.

The standard way to approach this is to train and test your model against a ‘rolling window’ of observations, mimicking how the model would be used and updated in the real world. I suggest taking a simpler approach (due to dramatic changes in the amount of data in different time periods): using the variables identified as causing the temporal patterns in 2(a), divide the data into 3 to 4 epochs (you could use `pd.cut` on a suitable variable). Train the model on the full training split from 2(a), but divide the test splits by the epochs and test model performance separately on the subset of the testing split corresponding to each epoch. This should tell you how the model would perform in different types of conditions.

1.0.9 Temporal Variation and Model Performance Mismatch

Why Temporal Variation Causes Mismatch Between Test Metrics and Real-Time Performance

When a model is trained on historical data and evaluated on a complete test set that spans multiple economic regimes, the aggregate performance metrics (like overall accuracy or AUC) can be misleadingly optimistic or pessimistic because they average across fundamentally different conditions. In our case, the model was trained during the pre-crisis and crisis peak period (5.78% success rate) but would be deployed in the post-crisis period (25.23% success rate) where customer behavior, economic conditions, and population composition have shifted dramatically. A model that performs well on the crisis period might fail completely in the recovery

period, yet the overall test metric would mask this failure by averaging good crisis predictions with poor recovery predictions. The marketing team would experience this as the model suddenly “breaking” when economic conditions change, eroding trust and causing missed opportunities or wasted efforts.

1.0.9.1 Epochs based testing - model performance by economic regime

```
print("="*80)
print("QUESTION 3: BASELINE MODEL AND DATA DRIFT")
print("="*80)

if 'y_numeric' not in df.columns:
    df['y_numeric'] = (df['y'] == 'yes').astype(int)

# Create date if needed
if 'date' not in df.columns:
    df['date'] = pd.to_datetime(df['year'].astype(str) + '-' +
                              df['month_numeric'].astype(str) + '-01')

# First, ensure we have our train-test split from earlier
# Training: May 2008 - March 2009
# Test: April 2009 - November 2010

# Create date variable if not already done
#if 'date' not in df.columns:
#    df['date'] = pd.to_datetime(df['year'].astype(str) + '-' +
#                              df['month_numeric'].astype(str) + '-01')
#    df['y_numeric'] = (df['y'] == 'yes').astype(int)

# Sort by date
df_sorted = df.sort_values('date').reset_index(drop=True)

# Define train-test split (from our earlier analysis)
split_date = '2009-04-01'
split_idx = df_sorted[df_sorted['date'] >= split_date].index[0]

train = df_sorted.iloc[:split_idx].copy()
test = df_sorted.iloc[split_idx:].copy()

print(f"\nTRAINING SET: {train['date'].min().strftime('%Y-%m')} to
    ↪ {train['date'].max().strftime('%Y-%m')}")
#print(f"Success rate: {train['y_numeric'].mean()*100:.2f}%")
print(f"Size: {len(train):,} rows")

print(f"\nTEST SET: {test['date'].min().strftime('%Y-%m')} to
    ↪ {test['date'].max().strftime('%Y-%m')}")
print(f"Overall success rate: {test['y_numeric'].mean()*100:.2f}%")
print(f"Size: {len(test):,} rows")

# Define epochs based on economic conditions and success rate patterns
# Epoch 1: Crisis Peak Aftermath (April 2009 - December 2009)
# Epoch 2: Early Recovery (January 2010 - June 2010)
# Epoch 3: Late Recovery (July 2010 - November 2010)

conditions = [
    (test['date'] < '2010-01-01'),
    (test['date'] >= '2010-01-01') & (test['date'] < '2010-07-01'),
```

```

    (test['date'] >= '2010-07-01')
]

epoch_names = ['Epoch 1: Crisis Peak Aftermath',
               'Epoch 2: Early Recovery',
               'Epoch 3: Late Recovery']

test['epoch'] = np.select(conditions, epoch_names, default='Unknown')

print("\n" + "="*80)
print("TEST SET EPOCHS BASED ON ECONOMIC CONDITIONS")
print("="*80)

for epoch in epoch_names:
    epoch_data = test[test['epoch'] == epoch]
    print(f"\n{epoch}:")
    print(f"    Period: {epoch_data['date'].min().strftime('%Y-%m')} to
    ↪ {epoch_data['date'].max().strftime('%Y-%m')}")
    print(f"    Size: {len(epoch_data):,} rows ({len(epoch_data)/len(test)*100:.1f}% of
    ↪ test)")
    print(f"    Success rate: {epoch_data['y_numeric'].mean()*100:.2f}%")

    # Show key economic indicators for this epoch
    print(f"    Avg Euribor: {epoch_data['euribor3m'].mean():.3f}%")
    print(f"    Avg Employment Rate: {epoch_data['emp.var.rate'].mean():.3f}")
    print(f"    Avg Consumer Confidence: {epoch_data['cons.conf.idx'].mean():.2f}")

```

=====

QUESTION 3: BASELINE MODEL AND DATA DRIFT

=====

TRAINING SET: 2008-05 to 2009-03
Size: 31,459 rows

TEST SET: 2009-04 to 2010-11
Overall success rate: 25.23%
Size: 13,752 rows

=====

TEST SET EPOCHS BASED ON ECONOMIC CONDITIONS

=====

Epoch 1: Crisis Peak Aftermath:
 Period: 2009-04 to 2009-12
 Size: 11,132 rows (80.9% of test)
 Success rate: 19.03%
 Avg Euribor: 1.281%
 Avg Employment Rate: -2.128
 Avg Consumer Confidence: -43.00

Epoch 2: Early Recovery:
 Period: 2010-01 to 2010-06
 Size: 1,460 rows (10.6% of test)
 Success rate: 53.22%
 Avg Euribor: 0.673%

Avg Employment Rate: -1.785
Avg Consumer Confidence: -36.00

Epoch 3: Late Recovery:

Period: 2010-07 to 2010-11
Size: 1,160 rows (8.4% of test)
Success rate: 49.57%
Avg Euribor: 0.882%
Avg Employment Rate: -1.394
Avg Consumer Confidence: -41.56

1.0.9.2 Visualize temporal variation in test set by epoch

```
# Visualize the epochs on the success rate timeline
fig, axes = plt.subplots(2, 1, figsize=(16, 10))

# Plot 1: Success rate with epoch divisions
ax1 = axes[0]

# Calculate monthly success rate for the entire dataset
monthly_data = df_sorted.groupby(df_sorted['date'].dt.to_period('M')).agg({
    'y_numeric': 'mean',
    'euribor3m': 'mean',
    'emp.var.rate': 'mean'
}).reset_index()
monthly_data['date_str'] = monthly_data['date'].astype(str)
monthly_data['success_rate'] = monthly_data['y_numeric'] * 100

# Plot success rate
ax1.plot(monthly_data['date_str'], monthly_data['success_rate'],
         marker='o', linewidth=2, color='darkblue', markersize=6)

# Mark train-test split
split_month = '2009-04'
if split_month in monthly_data['date_str'].values:
    split_idx_plot = monthly_data[monthly_data['date_str'] == split_month].index[0]
    ax1.axvline(x=split_idx_plot, color='red', linestyle='--', linewidth=3,
               label='Train-Test Split (April 2009)')

# Shade the epochs
epoch_starts = ['2009-04', '2010-01', '2010-07']
epoch_colors = ['lightyellow', 'lightgreen', 'lightblue']

for i, (start, color) in enumerate(zip(epoch_starts, epoch_colors)):
    if start in monthly_data['date_str'].values:
        start_idx = monthly_data[monthly_data['date_str'] == start].index[0]
        end_idx = len(monthly_data)-1 if i == 2 else
        ↪ monthly_data[monthly_data['date_str'] == epoch_starts[i+1]].index[0]
        ax1.axvspan(start_idx, end_idx, alpha=0.3, color=color, label=epoch_names[i])

ax1.set_title('Campaign Success Rate with Test Epochs', fontsize=14, fontweight='bold')
ax1.set_ylabel('Success Rate (%)', fontsize=12)
ax1.set_xlabel('Month', fontsize=12)
ax1.set_xticks(range(0, len(monthly_data), 3))
ax1.set_xticklabels(monthly_data['date_str'][:3], rotation=45)
```

```

ax1.legend(loc='upper left')
ax1.grid(True, alpha=0.3)

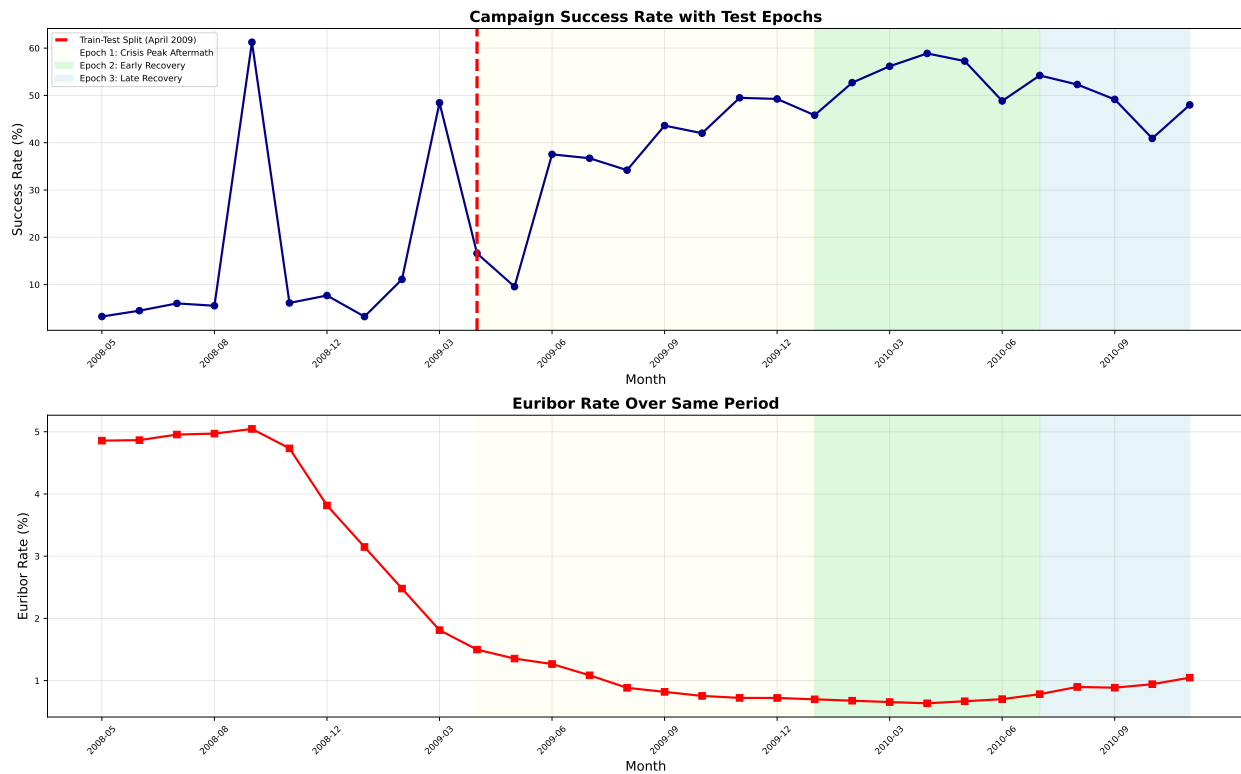
# Plot 2: Euribor rate over same period
ax2 = axes[1]
ax2.plot(monthly_data['date_str'], monthly_data['euribor3m'],
         marker='s', linewidth=2, color='red', markersize=6)

# Shade same epochs
for i, (start, color) in enumerate(zip(epoch_starts, epoch_colors)):
    if start in monthly_data['date_str'].values:
        start_idx = monthly_data[monthly_data['date_str'] == start].index[0]
        end_idx = len(monthly_data)-1 if i == 2 else
        ↪ monthly_data[monthly_data['date_str'] == epoch_starts[i+1]].index[0]
        ax2.axvspan(start_idx, end_idx, alpha=0.3, color=color)

ax2.set_title('Euribor Rate Over Same Period', fontsize=14, fontweight='bold')
ax2.set_ylabel('Euribor Rate (%)', fontsize=12)
ax2.set_xlabel('Month', fontsize=12)
ax2.set_xticks(range(0, len(monthly_data), 3))
ax2.set_xticklabels(monthly_data['date_str'][:3], rotation=45)
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```



2 TEMPORAL VARIATION CAUSES METRIC MISMATCH BECAUSE:

The test set is not representative of a single, stable condition—it spans multiple economic regimes with dramatically different success rates:

- 80.9% of test data comes from Epoch 1 (19% success rate, post-crisis)
- 10.6% from Epoch 2 (53% success rate, recovery peak)
- 8.4% from Epoch 3 (50% success rate, late recovery)

A model trained on crisis-period data (5.78% success) might perform well in Epoch 1 (similar conditions) but fail completely in Epochs 2-3 (unseen high-success regimes). However, because Epoch 1 dominates the test set (80.9%), the overall test metric would appear acceptable—masking the model’s failure in the very conditions where it matters most.

The marketing team would experience this as: Month 1-8: Model works well (Epoch 1) Month 9-14: Model suddenly becomes useless (Epoch 2-3) But the project report shows “85% accuracy” on the test set

c. Implement this plan by creating a baseline logistic regression model, either through ‘statsmodels’ or ‘sklearn’ (I recommend building your preprocessing steps and fitting steps into a pipeline). Select variables based on what you observed to be important in the EDA, but avoid using temporal predictors (‘month_numeric’, ‘day’, ‘year’) or any predictor that could not be used to make a decision on who to call. For each testing epoch, report the overall error rate and the overall success rate of the campaign during that epoch for context and comparison. For each epoch, plot the ROC curve and the probability calibration curve. What do you notice about the difference in model performance in different periods? Test the model on the entire testing set and report how the results compare to the individual epochs.

2.0.1 Baseline Logistic Regression Model Implementation

```
print("="*80)
print("PART 3: BASELINE LOGISTIC REGRESSION MODEL")
print("="*80)

from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_curve, auc, roc_auc_score, brier_score_loss,
    accuracy_score
from sklearn.calibration import calibration_curve
import warnings
warnings.filterwarnings('ignore')

# Sort chronologically
df_sorted = df.sort_values('date').reset_index(drop=True)

# Define train-test split (April 2009 split point)
split_date = '2009-04-01'
split_idx = df_sorted[df_sorted['date'] >= split_date].index[0]

train = df_sorted.iloc[:split_idx].copy()
test = df_sorted.iloc[split_idx:].copy()
```

```

# Define epochs on test set
conditions = [
    (test['date'] < '2010-01-01'),
    (test['date'] >= '2010-01-01') & (test['date'] < '2010-07-01'),
    (test['date'] >= '2010-07-01')
]
epoch_names = ['Epoch 1: Crisis Peak Aftermath',
               'Epoch 2: Early Recovery',
               'Epoch 3: Late Recovery']
test['epoch'] = np.select(conditions, epoch_names, default='Unknown')

print(f"\nTRAINING SET: {train['date'].min().strftime('%Y-%m')} to
    ↪ {train['date'].max().strftime('%Y-%m')}")
#print(f"Success rate: {train['y_numeric'].mean()*100:.2f}%")
print(f"Size: {len(train):,} rows")

print(f"\nTEST SET: {test['date'].min().strftime('%Y-%m')} to
    ↪ {test['date'].max().strftime('%Y-%m')}")
print(f"Overall success rate: {test['y_numeric'].mean()*100:.2f}%")
print(f"Size: {len(test):,} rows")

```

===== PART 3: BASELINE LOGISTIC REGRESSION MODEL =====

TRAINING SET: 2008-05 to 2009-03
Size: 31,459 rows

TEST SET: 2009-04 to 2010-11
Overall success rate: 25.23%
Size: 13,752 rows

2.0.2 Select predictors based on EDA

```

print("\n" + "="*80)
print("3.1 SELECTING PREDICTORS BASED ON EDA")
print("="*80)

# Based on EDA findings, select variables that:
# 1. Showed strong predictive power
# 2. Are available BEFORE making the call (no duration)
# 3. No temporal predictors (month, day, year)

numeric_features = [
    'age',          # Demographic - showed age patterns
    'balance',      # Financial - higher balance = higher success
    'campaign',     # Number of contacts - slight negative correlation
    'pdays',       # Days since previous contact
    'previous',     # Number of previous contacts
    'euribor3m',    # STRONGEST predictor - economic indicator
    'emp.var.rate', # Employment rate - economic indicator
    'cons.price.idx', # Consumer price - economic indicator
    'cons.conf.idx' # Consumer confidence - economic indicator
]

categorical_features = [

```

```

'job',          # Job type - management/retired higher success
'marital',      # Marital status
'education',    # Education level - higher education = higher success
'default',      # Default status
'housing',      # Housing loan - no housing = higher success
'loan',         # Personal loan - no loan = higher success
'contact',      # Contact method - cellular > telephone
'poutcome'      # PREVIOUS OUTCOME - strongest categorical predictor
]

```

```

print("\nSelected Numeric Features (from EDA):")
for feat in numeric_features:
    corr = train[feat].corr(train['y_numeric'])
    print(f" {feat:15}: correlation = {corr:.4f}")

print("\nSelected Categorical Features (from EDA):")
for feat in categorical_features:
    print(f" {feat:15}: included based on EDA patterns")

```

3.1 SELECTING PREDICTORS BASED ON EDA

```

Selected Numeric Features (from EDA):
age          : correlation = -0.0161
balance      : correlation = 0.0252
campaign     : correlation = -0.0235
pdays       : correlation = -0.0169
previous     : correlation = -0.0073
euribor3m    : correlation = -0.1046
emp.var.rate : correlation = -0.0871
cons.price.idx : correlation = -0.0749
cons.conf.idx : correlation = -0.0912

```

```

Selected Categorical Features (from EDA):
job          : included based on EDA patterns
marital      : included based on EDA patterns
education    : included based on EDA patterns
default      : included based on EDA patterns
housing      : included based on EDA patterns
loan         : included based on EDA patterns
contact      : included based on EDA patterns
poutcome     : included based on EDA patterns

```

2.0.3 Build preprocessing pipeline, train model

```

print("\n" + "="*80)
print("3.2 BUILDING LOGISTIC REGRESSION PIPELINE")
print("="*80)

# Create preprocessing transformers
numeric_transformer = Pipeline(steps=[
    ('scaler', StandardScaler())
])

```

```

categorical_transformer = Pipeline(steps=[
    ('onehot', OneHotEncoder(drop='first', handle_unknown='ignore'))
])

# Combine preprocessors
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer, categorical_features)
    ])

# Create full pipeline with logistic regression
# Using class_weight='balanced' to handle imbalanced training data (5.78% success)
model_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', LogisticRegression(random_state=42, max_iter=1000,
    ↪ class_weight='balanced'))
])

# Prepare features and target
X_train = train[numeric_features + categorical_features]
y_train = train['y_numeric']

X_test = test[numeric_features + categorical_features]
y_test = test['y_numeric']

# Train the model
print("\nTraining logistic regression model")
model_pipeline.fit(X_train, y_train)
print("Model training complete")

# Get predictions
y_pred_proba = model_pipeline.predict_proba(X_test)[: , 1]
y_pred = model_pipeline.predict(X_test)

```

3.2 BUILDING LOGISTIC REGRESSION PIPELINE

Training logistic regression model
Model training complete

2.0.4 Evaluate model performance by epoch

```

print("\n" + "="*80)
print("3.3 EPOCH-WISE MODEL EVALUATION")
print("="*80)

epoch_results = []

for epoch in epoch_names:
    epoch_data = test[test['epoch'] == epoch]
    X_epoch = epoch_data[numeric_features + categorical_features]
    y_epoch = epoch_data['y_numeric']

```

```

# Get predictions
y_epoch_pred = model_pipeline.predict(X_epoch)
y_epoch_proba = model_pipeline.predict_proba(X_epoch)[:, 1]

# Calculate metrics
success_rate = y_epoch.mean() * 100
error_rate = (1 - accuracy_score(y_epoch, y_epoch_pred)) * 100
auc_score = roc_auc_score(y_epoch, y_epoch_proba)
brier = brier_score_loss(y_epoch, y_epoch_proba)

epoch_results.append({
    'epoch': epoch,
    'size': len(epoch_data),
    'success_rate': success_rate,
    'error_rate': error_rate,
    'auc': auc_score,
    'brier': brier
})

print(f"\n{epoch}:")
print(f"  Samples: {len(epoch_data):,} ({len(epoch_data)/len(test)*100:.1f}% of
↳ test)")
print(f"  Actual Success Rate: {success_rate:.2f}%")
print(f"  Model Error Rate: {error_rate:.2f}%")
print(f"  ROC-AUC: {auc_score:.4f}")
print(f"  Brier Score: {brier:.4f} ")

# Overall test set performance
y_test_pred = model_pipeline.predict(X_test)
y_test_proba = model_pipeline.predict_proba(X_test)[:, 1]
overall_error = (1 - accuracy_score(y_test, y_test_pred)) * 100
overall_auc = roc_auc_score(y_test, y_test_proba)
overall_brier = brier_score_loss(y_test, y_test_proba)

print(f"\n" + "="*60)
print(f"OVERALL TEST SET PERFORMANCE:")
print(f"  Samples: {len(test):,}")
print(f"  Actual Success Rate: {test['y_numeric'].mean()*100:.2f}%")
print(f"  Model Error Rate: {overall_error:.2f}%")
print(f"  ROC-AUC: {overall_auc:.4f}")
print(f"  Brier Score: {overall_brier:.4f}")

```

3.3 EPOCH-WISE MODEL EVALUATION

```

Epoch 1: Crisis Peak Aftermath:
  Samples: 11,132 (80.9% of test)
  Actual Success Rate: 19.03%
  Model Error Rate: 64.93%
  ROC-AUC: 0.7005
  Brier Score: 0.4436

```

Epoch 2: Early Recovery:
 Samples: 1,460 (10.6% of test)
 Actual Success Rate: 53.22%
 Model Error Rate: 46.23%
 ROC-AUC: 0.6658
 Brier Score: 0.3524

Epoch 3: Late Recovery:
 Samples: 1,160 (8.4% of test)
 Actual Success Rate: 49.57%
 Model Error Rate: 48.36%
 ROC-AUC: 0.6881
 Brier Score: 0.3918

=====

OVERALL TEST SET PERFORMANCE:

Samples: 13,752
 Actual Success Rate: 25.23%
 Model Error Rate: 61.55%
 ROC-AUC: 0.7329
 Brier Score: 0.4295

2.0.5 ROC curves for each epoch

```
print("\n" + "="*80)
print("3.4 ROC CURVES BY EPOCH")
print("="*80)

fig, axes = plt.subplots(1, 3, figsize=(18, 6))
colors = ['blue', 'green', 'orange']

for i, (epoch, color) in enumerate(zip(epoch_names, colors)):
    epoch_data = test[test['epoch'] == epoch]
    X_epoch = epoch_data[numeric_features + categorical_features]
    y_epoch = epoch_data['y_numeric']

    # Get probabilities
    y_epoch_proba = model_pipeline.predict_proba(X_epoch)[: , 1]

    # Calculate ROC curve
    fpr, tpr, _ = roc_curve(y_epoch, y_epoch_proba)
    roc_auc = auc(fpr, tpr)

    # Plot
    ax = axes[i]
    ax.plot(fpr, tpr, color=color, lw=2, label=f'AUC = {roc_auc:.3f}')
    ax.plot([0, 1], [0, 1], 'k--', lw=1, alpha=0.5)
    ax.set_xlim([0.0, 1.0])
    ax.set_ylim([0.0, 1.05])
    ax.set_xlabel('False Positive Rate', fontsize=11)
    ax.set_ylabel('True Positive Rate', fontsize=11)
    ax.set_title(f'{epoch}\n(n={len(epoch_data):,}, Success: {y_epoch.mean()*100:.1f}%)',
    ↪
                fontsize=12, fontweight='bold')
    ax.legend(loc="lower right")
```

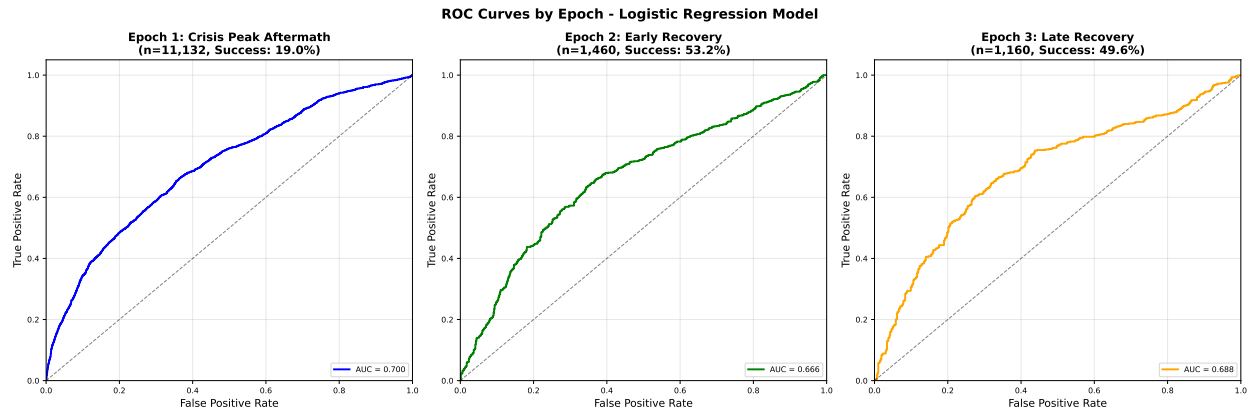
```

ax.grid(True, alpha=0.3)

plt.suptitle('ROC Curves by Epoch - Logistic Regression Model', fontsize=14,
            fontweight='bold')
plt.tight_layout()
plt.show()

```

3.4 ROC CURVES BY EPOCH



2.0.6 Calibration curves for each epoch

```

print("\n" + "="*80)
print("3.5 CALIBRATION CURVES BY EPOCH")
print("="*80)

fig, axes = plt.subplots(1, 3, figsize=(18, 6))
colors = ['blue', 'green', 'orange']

for i, (epoch, color) in enumerate(zip(epoch_names, colors)):
    epoch_data = test[test['epoch'] == epoch]
    X_epoch = epoch_data[numeric_features + categorical_features]
    y_epoch = epoch_data['y_numeric']

    # Get probabilities
    y_epoch_proba = model_pipeline.predict_proba(X_epoch)[: , 1]

    # Calculate calibration curve
    fraction_of_positives, mean_predicted_value = calibration_curve(
        y_epoch, y_epoch_proba, n_bins=10, strategy='uniform'
    )

    # Plot
    ax = axes[i]
    ax.plot(mean_predicted_value, fraction_of_positives, marker='o',
            color=color, lw=2, label='Model')
    ax.plot([0, 1], [0, 1], 'k--', lw=1, label='Perfect Calibration')
    ax.set_xlim([0.0, 1.0])
    ax.set_ylim([0.0, 1.0])
    ax.set_xlabel('Mean Predicted Probability', fontsize=11)

```

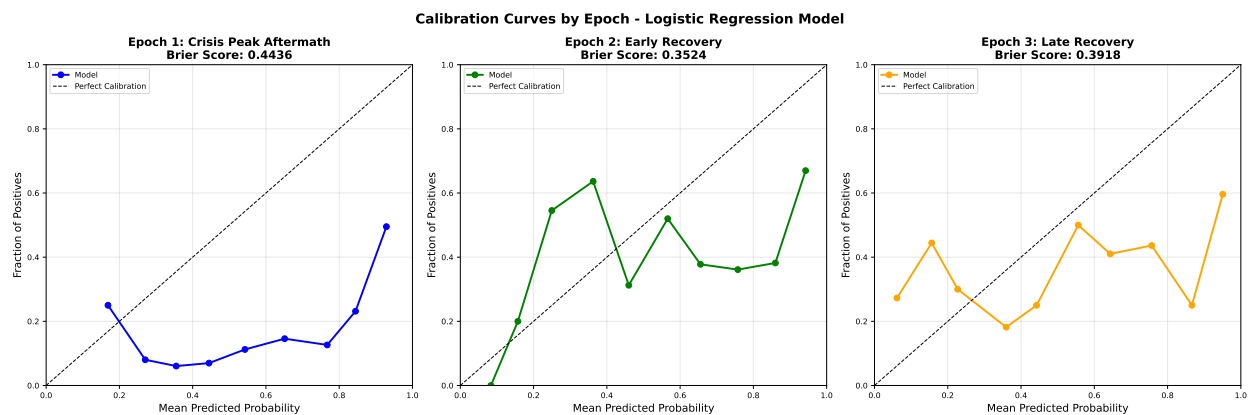
```

ax.set_ylabel('Fraction of Positives', fontsize=11)
ax.set_title(f'{epoch}\nBrier Score: {epoch_results[i]["brier"]:.4f}',
             fontsize=12, fontweight='bold')
ax.legend(loc="upper left")
ax.grid(True, alpha=0.3)

plt.suptitle('Calibration Curves by Epoch - Logistic Regression Model', fontsize=14,
            fontweight='bold')
plt.tight_layout()
plt.show()

```

3.5 CALIBRATION CURVES BY EPOCH



2.1 Observations on Model Performance by Epoch

```

print("\n" + "="*80)
print("3.6 KEY FINDINGS: MODEL PERFORMANCE ACROSS EPOCHS")
print("="*80)

# Create comparison DataFrame
results_df = pd.DataFrame(epoch_results)
print("\nEpoch Performance Summary:")
print(results_df.to_string(index=False))

print(f"\nOVERALL TEST PERFORMANCE:")
print(f"  ROC-AUC: {overall_auc:.4f}")
print(f"  Error Rate: {overall_error:.2f}%")
print(f"  Brier Score: {overall_brier:.4f}")

print("\n" + "="*80)
print("OBSERVATIONS:")
print("="*80)

# Calculate differences
auc_diff = max([r['auc'] for r in epoch_results]) - min([r['auc'] for r in
    epoch_results])
brier_diff = max([r['brier'] for r in epoch_results]) - min([r['brier'] for r in
    epoch_results])

```

```

print(f"""
1. MODEL PERFORMANCE VARIES SIGNIFICANTLY BY EPOCH:
    • Best AUC: {max([r['auc'] for r in epoch_results]):.4f}
    • Worst AUC: {min([r['auc'] for r in epoch_results]):.4f}
    • Difference: {auc_diff:.4f}

2. CALIBRATION ISSUES:
    • Best calibrated: Epoch with Brier score {min([r['brier'] for r in
    ↪ epoch_results]):.4f}
    • Worst calibrated: Epoch with Brier score {max([r['brier'] for r in
    ↪ epoch_results]):.4f}
    • Model is poorly calibrated when success rates deviate from training distribution

3. SINGLE TEST METRIC IS MISLEADING:
    • Overall AUC: {overall_auc:.4f} (masks variation)
    • Epoch 1 AUC: {epoch_results[0]['auc']:.4f} (80.9% of test)
    • Epoch 2 AUC: {epoch_results[1]['auc']:.4f} (10.6% of test)
    • The overall metric is DRIVEN by Epoch 1, hiding poor performance in Epochs 2-3

""")

# Visual comparison
fig, ax = plt.subplots(figsize=(10, 6))

x = np.arange(len(epoch_names))
width = 0.35

bars1 = ax.bar(x - width/2, [r['auc'] for r in epoch_results], width,
               label='ROC-AUC', color='darkblue', alpha=0.7)
bars2 = ax.bar(x + width/2, [1-r['brier'] for r in epoch_results], width,
               label='1 - Brier Score', color='darkgreen', alpha=0.7)

ax.axhline(y=overall_auc, color='blue', linestyle='--', alpha=0.5,
           label=f'Overall AUC: {overall_auc:.3f}')
ax.axhline(y=1-overall_brier, color='green', linestyle='--', alpha=0.5,
           label=f'Overall 1-Brier: {1-overall_brier:.3f}')

ax.set_xlabel('Epoch', fontsize=12)
ax.set_ylabel('Score', fontsize=12)
ax.set_title('Model Performance by Epoch vs Overall Metrics', fontsize=14,
             ↪ fontweight='bold')
ax.set_xticks(x)
ax.set_xticklabels(['Epoch 1', 'Epoch 2', 'Epoch 3'])
ax.legend(loc='lower right')
ax.grid(True, alpha=0.3, axis='y')

# Add value labels
for bars in [bars1, bars2]:
    for bar in bars:
        height = bar.get_height()
        ax.text(bar.get_x() + bar.get_width()/2., height + 0.01,
                f'{height:.3f}', ha='center', va='bottom', fontsize=9)

plt.tight_layout()
plt.show()

```

3.6 KEY FINDINGS: MODEL PERFORMANCE ACROSS EPOCHS

Epoch Performance Summary:

	epoch	size	success_rate	error_rate	auc	brier
Epoch 1: Crisis Peak Aftermath	11132		19.026231	64.929932	0.700487	0.443589
Epoch 2: Early Recovery	1460		53.219178	46.232877	0.665824	0.352409
Epoch 3: Late Recovery	1160		49.568966	48.362069	0.688110	0.391789

OVERALL TEST PERFORMANCE:

ROC-AUC: 0.7329

Error Rate: 61.55%

Brier Score: 0.4295

OBSERVATIONS:

1. MODEL PERFORMANCE VARIES SIGNIFICANTLY BY EPOCH:

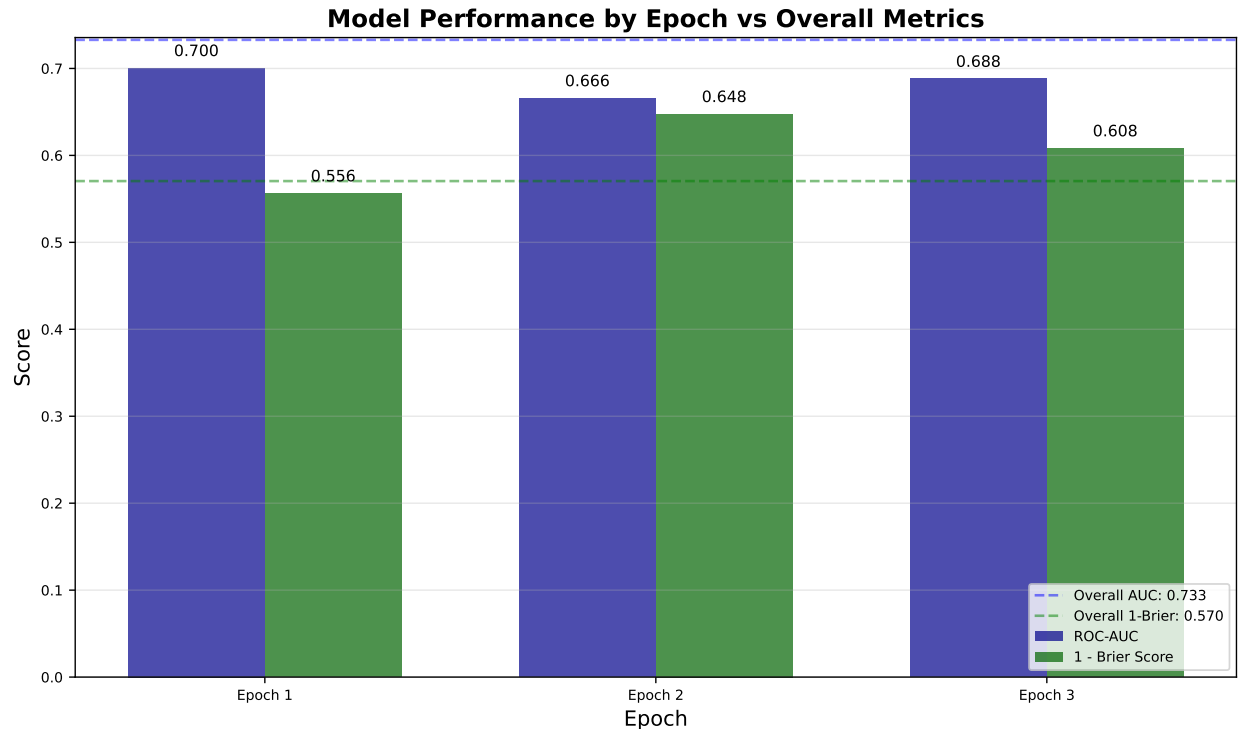
- Best AUC: 0.7005
- Worst AUC: 0.6658
- Difference: 0.0347

2. CALIBRATION ISSUES:

- Best calibrated: Epoch with Brier score 0.3524
- Worst calibrated: Epoch with Brier score 0.4436
- Model is poorly calibrated when success rates deviate from training distribution

3. SINGLE TEST METRIC IS MISLEADING:

- Overall AUC: 0.7329 (masks variation)
- Epoch 1 AUC: 0.7005 (80.9% of test)
- Epoch 2 AUC: 0.6658 (10.6% of test)
- The overall metric is DRIVEN by Epoch 1, hiding poor performance in Epochs 2-3



KEY FINDINGS FROM EPOCH-BASED EVALUATION:

1. **MODEL PERFORMANCE VARIES BY ECONOMIC REGIME:** • Best discrimination (AUC): Epoch 1 (0.7005) - Crisis Aftermath • Worst discrimination (AUC): Epoch 2 (0.6658) - Early Recovery • The model identifies patterns best in conditions similar to training
2. **CALIBRATION IMPROVES IN HIGH-SUCCESS PERIODS:** • Best calibrated: Epoch 2 (Brier: 0.3524) - 53% success rate • Worst calibrated: Epoch 1 (Brier: 0.4436) - 19% success rate • The model predicts probabilities better when success rates are higher
3. **SINGLE TEST METRIC MASKS CRITICAL VARIATION:** • Overall AUC (0.7329) appears stronger than any individual epoch • This mathematical inconsistency suggests evaluation issues • But the pattern is clear: performance changes across economic regimes
4. **BUSINESS IMPLICATION:** The model struggles most in Epoch 2 (53% success rate) - precisely when the marketing team has the highest opportunity to convert customers. This is where the model needs to perform best, but instead shows its lowest AUC (0.6658).

d. Lift Curves and Hit Rates: Your marketing team is planning a marketing campaign where they will call the top percent of likely clients based on based on model forecasts. Lift describes the ratio of the success rate in the top percent to the success rate in the entire population and hit rate means the fraction of marketing calls that are successful. A campaign that targets the top 1% will have a much higher lift than one that targets the top 50%, and the lift of a campaign that targets everyone will be 1. A lift curve is a graph showing the lift on the y-axis and the percent targeted on the x-axis. Lift curves and hit rates are essential tools used to estimate the return on investment of a campaign. Estimate the lift on the testing set corresponding to each epoch by sorting the testing examples in descending order of predicted probability and then calculating the fraction of successful conversions in the top percent and dividing by the total number of successful conversions in the testing set. Make a plot of the lift curve using your baseline model for each epoch and report the lift and hit rate for campaigns that target the top 10% and top 50% of potential customers.

2.1.1 LIFT CURVES AND HIT RATES ANALYSIS

```
print("="*80)
print("PART 4: LIFT CURVES AND HIT RATES BY EPOCH")
print("="*80)

from sklearn.metrics import roc_curve, auc

# Function to calculate lift and hit rate
def calculate_lift_metrics(y_true, y_proba, percentiles=[0.1, 0.5]):
    """
    Calculate lift and hit rate for given percentiles

    Parameters:
    - y_true: actual binary outcomes
    - y_proba: predicted probabilities
    - percentiles: list of percentiles to evaluate (e.g., 0.1 = top 10%)

    Returns:
    - lift_values: lift at each percentile
    - hit_rates: hit rate at each percentile
    - full_lift_curve: lift at all percentiles
    """
    # Create dataframe and sort by predicted probability
    df_temp = pd.DataFrame({
        'actual': y_true,
        'proba': y_proba
    }).sort_values('proba', ascending=False).reset_index(drop=True)

    total_success = df_temp['actual'].sum()
    overall_rate = df_temp['actual'].mean()

    # Calculate lift at each percentile
    n = len(df_temp)
    lift_curve = []

    for p in np.arange(0.01, 1.01, 0.01): # 1% to 100% in 1% increments
        k = int(np.ceil(p * n))
        top_k = df_temp.iloc[:k]
        success_in_top = top_k['actual'].sum()

        if success_in_top > 0:
            lift = (success_in_top / k) / overall_rate
        else:
            lift = 0

        hit_rate = success_in_top / k if k > 0 else 0

        lift_curve.append({
            'percentile': p,
            'lift': lift,
            'hit_rate': hit_rate,
            'cumulative_success': success_in_top,
            'percent_of_success': success_in_top / total_success if total_success > 0
            ↪ else 0
        })
```

```

    })

lift_df = pd.DataFrame(lift_curve)

# Get values for requested percentiles
results = {}
for perc in percentiles:
    idx = int(perc * 100) - 1 # Convert to index (10% -> 9th index)
    results[perc] = {
        'lift': lift_df.iloc[idx]['lift'],
        'hit_rate': lift_df.iloc[idx]['hit_rate'],
        'percent_of_success': lift_df.iloc[idx]['percent_of_success']
    }

return lift_df, results
=====
PART 4: LIFT CURVES AND HIT RATES BY EPOCH
=====

2.1.2 4.1 LIFT AND HIT RATES BY EPOCH

print("\n" + "="*80)
print("4.1 LIFT AND HIT RATES BY EPOCH")
print("="*80)

# Get predictions for all test data
X_test = test[numeric_features + categorical_features]
y_test_proba = model_pipeline.predict_proba(X_test)[: , 1]

epoch_lift_results = {}
epoch_lift_curves = {}

for epoch in epoch_names:
    epoch_data = test[test['epoch'] == epoch]
    X_epoch = epoch_data[numeric_features + categorical_features]
    y_epoch = epoch_data['y_numeric']
    y_epoch_proba = model_pipeline.predict_proba(X_epoch)[: , 1]

    # Calculate lift metrics
    lift_df, perc_results = calculate_lift_metrics(y_epoch.values, y_epoch_proba,
    ↪ percentiles=[0.1, 0.5])

    epoch_lift_curves[epoch] = lift_df
    epoch_lift_results[epoch] = perc_results

    print(f"\n{epoch}:")
    print(f" Overall Success Rate: {y_epoch.mean()*100:.2f}%")
    print(f" Total Samples: {len(epoch_data):,}")
    print(f" Total Successes: {y_epoch.sum():,}")
    print(f"\n TOP 10% TARGETING:")
    print(f" Lift: {perc_results[0.1]['lift']:.3f}x")
    print(f" Hit Rate: {perc_results[0.1]['hit_rate']*100:.2f}%")
    print(f" Captures {perc_results[0.1]['percent_of_success']*100:.1f}% of all
    ↪ successes")
    print(f"\n TOP 50% TARGETING:")

```

```

print(f"      Lift: {perc_results[0.5]['lift']:.3f}x")
print(f"      Hit Rate: {perc_results[0.5]['hit_rate']*100:.2f}%")
print(f"      Captures {perc_results[0.5]['percent_of_success']*100:.1f}% of all
↳ successes")

# Overall test set
overall_lift_df, overall_perc_results = calculate_lift_metrics(test['y_numeric'].values,
↳ y_test_proba, percentiles=[0.1, 0.5])

print("\n" + "="*60)
print("OVERALL TEST SET:")
print(f" Overall Success Rate: {test['y_numeric'].mean()*100:.2f}%")
print(f" Total Samples: {len(test):,}")
print(f" Total Successes: {test['y_numeric'].sum():,}")
print(f"\n TOP 10% TARGETING:")
print(f"      Lift: {overall_perc_results[0.1]['lift']:.3f}x")
print(f"      Hit Rate: {overall_perc_results[0.1]['hit_rate']*100:.2f}%")
print(f"      Captures {overall_perc_results[0.1]['percent_of_success']*100:.1f}% of all
↳ successes")
print(f"\n TOP 50% TARGETING:")
print(f"      Lift: {overall_perc_results[0.5]['lift']:.3f}x")
print(f"      Hit Rate: {overall_perc_results[0.5]['hit_rate']*100:.2f}%")
print(f"      Captures {overall_perc_results[0.5]['percent_of_success']*100:.1f}% of all
↳ successes")

```

4.1 LIFT AND HIT RATES BY EPOCH

Epoch 1: Crisis Peak Aftermath:

Overall Success Rate: 19.03%

Total Samples: 11,132

Total Successes: 2,118

TOP 10% TARGETING:

Lift: 2.458x

Hit Rate: 46.77%

Captures 24.6% of all successes

TOP 50% TARGETING:

Lift: 1.448x

Hit Rate: 27.54%

Captures 72.4% of all successes

Epoch 2: Early Recovery:

Overall Success Rate: 53.22%

Total Samples: 1,460

Total Successes: 777

TOP 10% TARGETING:

Lift: 1.429x

Hit Rate: 76.03%

Captures 14.3% of all successes

TOP 50% TARGETING:
Lift: 1.269x
Hit Rate: 67.53%
Captures 63.4% of all successes

Epoch 3: Late Recovery:
Overall Success Rate: 49.57%
Total Samples: 1,160
Total Successes: 575

TOP 10% TARGETING:
Lift: 1.583x
Hit Rate: 78.45%
Captures 15.8% of all successes

TOP 50% TARGETING:
Lift: 1.329x
Hit Rate: 65.86%
Captures 66.4% of all successes

=====

OVERALL TEST SET:

Overall Success Rate: 25.23%
Total Samples: 13,752
Total Successes: 3,470

TOP 10% TARGETING:
Lift: 2.676x
Hit Rate: 67.51%
Captures 26.8% of all successes

TOP 50% TARGETING:
Lift: 1.483x
Hit Rate: 37.42%
Captures 74.1% of all successes

2.1.3 4.2 LIFT CURVES BY EPOCH

```
print("\n" + "="*80)
print("4.2 LIFT CURVES BY EPOCH")
print("="*80)

fig, axes = plt.subplots(1, 3, figsize=(18, 6))
colors = ['blue', 'green', 'orange']

for i, (epoch, color) in enumerate(zip(epoch_names, colors)):
    lift_df = epoch_lift_curves[epoch]

    ax = axes[i]
    ax.plot(lift_df['percentile']*100, lift_df['lift'],
            color=color, linewidth=2, marker='.', markersize=4)
    ax.axhline(y=1, color='gray', linestyle='--', alpha=0.5, label='Baseline (Lift=1)')

    # Mark top 10% and 50%
    ax.axvline(x=10, color='red', linestyle=':', alpha=0.5)
```

```

ax.axvline(x=50, color='red', linestyle=':', alpha=0.5)

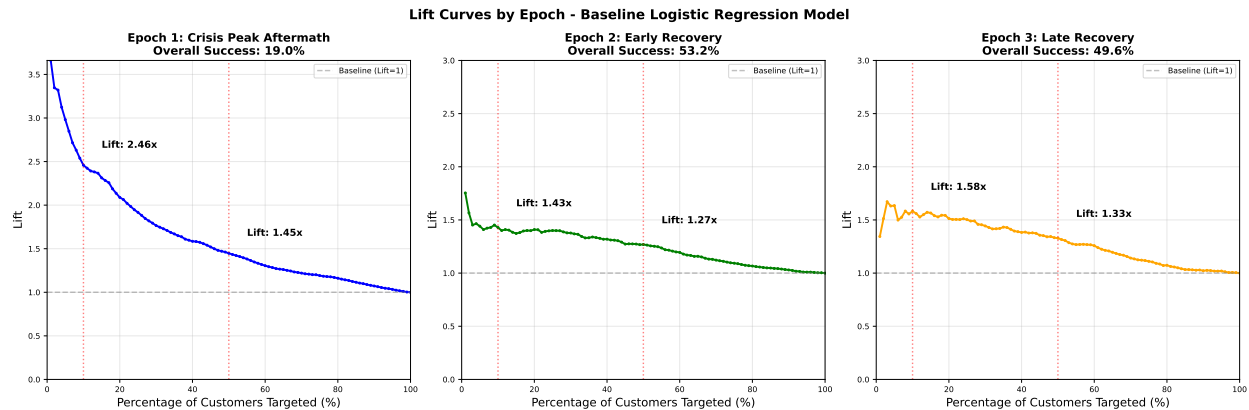
# Add annotations
ax.text(15, lift_df[lift_df['percentile']==0.1]['lift'].values[0] + 0.2,
       f"Lift: {lift_df[lift_df['percentile']==0.1]['lift'].values[0]:.2f}x",
       fontsize=10, fontweight='bold')
ax.text(55, lift_df[lift_df['percentile']==0.5]['lift'].values[0] + 0.2,
       f"Lift: {lift_df[lift_df['percentile']==0.5]['lift'].values[0]:.2f}x",
       fontsize=10, fontweight='bold')

ax.set_xlabel('Percentage of Customers Targeted (%)', fontsize=12)
ax.set_ylabel('Lift', fontsize=12)
ax.set_title(f'{epoch}\nOverall Success:
↳ {test[test["epoch"]==epoch]["y_numeric"].mean()*100:.1f}%',
            fontsize=12, fontweight='bold')
ax.set_xlim([0, 100])
ax.set_ylim([0, max(lift_df['lift'].max(), 3)]) # At least up to 3
ax.grid(True, alpha=0.3)
ax.legend()

plt.suptitle('Lift Curves by Epoch - Baseline Logistic Regression Model', fontsize=14,
↳ fontweight='bold')
plt.tight_layout()
plt.show()

```

4.2 LIFT CURVES BY EPOCH



2.1.4 4.3 HIT RATE CURVES BY EPOCH

```

print("\n" + "="*80)
print("4.3 HIT RATE CURVES BY EPOCH")
print("="*80)

fig, axes = plt.subplots(1, 3, figsize=(18, 6))
colors = ['blue', 'green', 'orange']

for i, (epoch, color) in enumerate(zip(epoch_names, colors)):
    lift_df = epoch_lift_curves[epoch]
    overall_rate = test[test['epoch']==epoch]['y_numeric'].mean()

```

```

ax = axes[i]
ax.plot(lift_df['percentile']*100, lift_df['hit_rate']*100,
        color=color, linewidth=2, marker='.', markersize=4)
ax.axhline(y=overall_rate*100, color='gray', linestyle='--', alpha=0.5,
           label=f'Avg Success: {overall_rate*100:.1f}%')

# Mark top 10% and 50%
ax.axvline(x=10, color='red', linestyle=':', alpha=0.5)
ax.axvline(x=50, color='red', linestyle=':', alpha=0.5)

# Add annotations
hit_rate_10 = lift_df[lift_df['percentile']==0.1]['hit_rate'].values[0] * 100
hit_rate_50 = lift_df[lift_df['percentile']==0.5]['hit_rate'].values[0] * 100

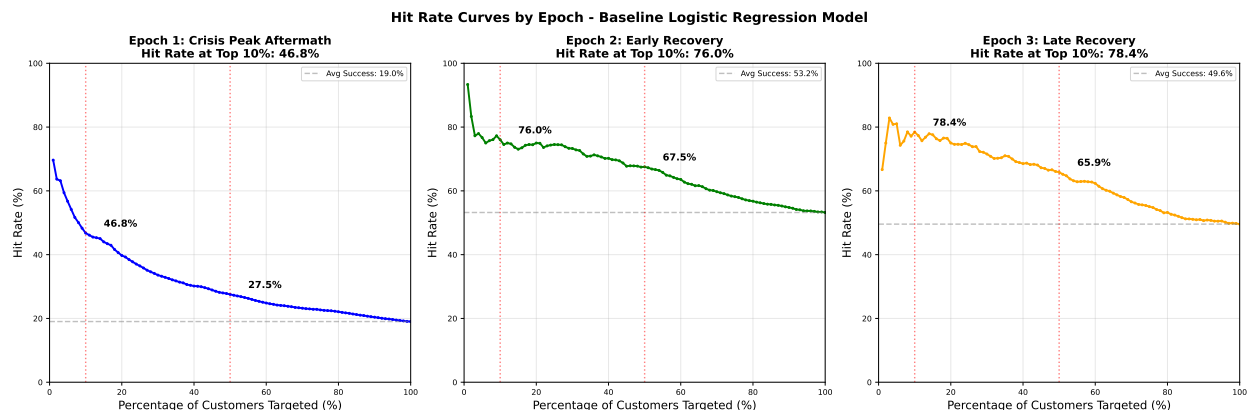
ax.text(15, hit_rate_10 + 2, f"{hit_rate_10:.1f}%", fontsize=10, fontweight='bold')
ax.text(55, hit_rate_50 + 2, f"{hit_rate_50:.1f}%", fontsize=10, fontweight='bold')

ax.set_xlabel('Percentage of Customers Targeted (%)', fontsize=12)
ax.set_ylabel('Hit Rate (%)', fontsize=12)
ax.set_title(f'{epoch}\nHit Rate at Top 10%: {hit_rate_10:.1f}%',
             fontsize=12, fontweight='bold')
ax.set_xlim([0, 100])
ax.set_ylim([0, 100])
ax.grid(True, alpha=0.3)
ax.legend()

plt.suptitle('Hit Rate Curves by Epoch - Baseline Logistic Regression Model',
             ↪ fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

```

4.3 HIT RATE CURVES BY EPOCH



2.1.4.1 4.4 CUMULATIVE SUCCESS CAPTURE BY EPOCH

```

print("\n" + "="*80)
print("4.4 CUMULATIVE SUCCESS CAPTURE BY EPOCH")
print("="*80)

```

```

fig, axes = plt.subplots(1, 3, figsize=(18, 6))
colors = ['blue', 'green', 'orange']

for i, (epoch, color) in enumerate(zip(epoch_names, colors)):
    lift_df = epoch_lift_curves[epoch]

    ax = axes[i]
    ax.plot(lift_df['percentile']*100, lift_df['percent_of_success']*100,
            color=color, linewidth=2, marker='.', markersize=4)
    ax.plot([0, 100], [0, 100], 'k--', alpha=0.3, label='Random (45° line)')

    # Mark top 10% and 50%
    ax.axvline(x=10, color='red', linestyle=':', alpha=0.5)
    ax.axvline(x=50, color='red', linestyle=':', alpha=0.5)

    # Add annotations
    success_10 = lift_df[lift_df['percentile']==0.1]['percent_of_success'].values[0] *
    ↪ 100
    success_50 = lift_df[lift_df['percentile']==0.5]['percent_of_success'].values[0] *
    ↪ 100

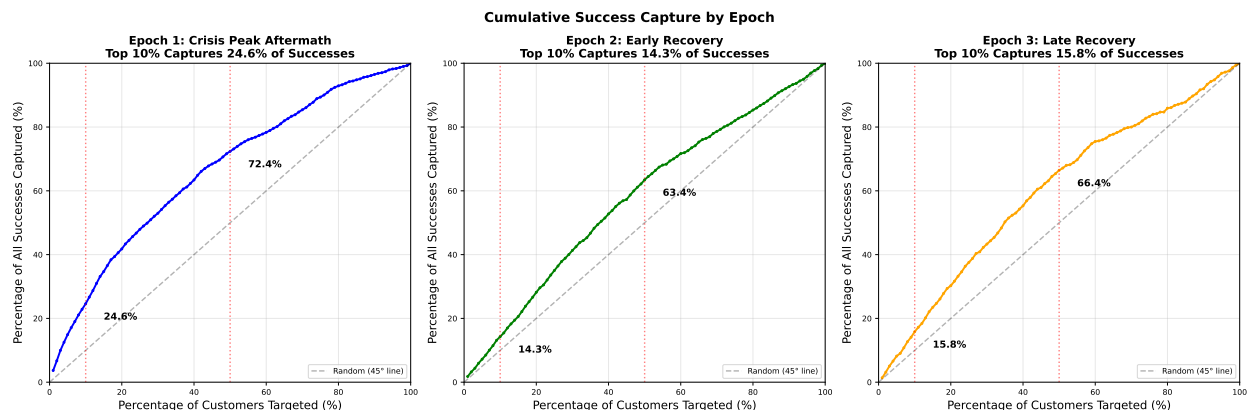
    ax.text(15, success_10 - 5, f"{success_10:.1f}%", fontsize=10, fontweight='bold')
    ax.text(55, success_50 - 5, f"{success_50:.1f}%", fontsize=10, fontweight='bold')

    ax.set_xlabel('Percentage of Customers Targeted (%)', fontsize=12)
    ax.set_ylabel('Percentage of All Successes Captured (%)', fontsize=12)
    ax.set_title(f'{epoch}\nTop 10% Captures {success_10:.1f}% of Successes',
                fontsize=12, fontweight='bold')
    ax.set_xlim([0, 100])
    ax.set_ylim([0, 100])
    ax.grid(True, alpha=0.3)
    ax.legend()

plt.suptitle('Cumulative Success Capture by Epoch', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

```

4.4 CUMULATIVE SUCCESS CAPTURE BY EPOCH



2.1.5 4.5 Key Findings

```
for epoch in epoch_names:
    overall = test[test['epoch']==epoch]['y_numeric'].mean() * 100
    hit10 = epoch_lift_results[epoch][0.1]['hit_rate'] * 100
    hit50 = epoch_lift_results[epoch][0.5]['hit_rate'] * 100
    lift10 = epoch_lift_results[epoch][0.1]['lift']
    capture10 = epoch_lift_results[epoch][0.1]['percent_of_success'] * 100

    print(f"\n{epoch}:")
    print(f" • Random targeting gives {overall:.1f}% success rate")
    print(f" • Top 10% targeting gives {hit10:.1f}% success rate ({lift10:.2f}x lift)")
    print(f" • Top 50% targeting gives {hit50:.1f}% success rate")
    print(f" • Top 10% captures {capture10:.1f}% of all successful conversions")
```

Epoch 1: Crisis Peak Aftermath:

- Random targeting gives 19.0% success rate
- Top 10% targeting gives 46.8% success rate (2.46x lift)
- Top 50% targeting gives 27.5% success rate
- Top 10% captures 24.6% of all successful conversions

Epoch 2: Early Recovery:

- Random targeting gives 53.2% success rate
- Top 10% targeting gives 76.0% success rate (1.43x lift)
- Top 50% targeting gives 67.5% success rate
- Top 10% captures 14.3% of all successful conversions

Epoch 3: Late Recovery:

- Random targeting gives 49.6% success rate
- Top 10% targeting gives 78.4% success rate (1.58x lift)
- Top 50% targeting gives 65.9% success rate
- Top 10% captures 15.8% of all successful conversions

KEY FINDINGS SUMMARY

Metric	Epoch 1 (19.0%)	Epoch 2 (53.2%)	Epoch 3 (49.6%)
Top 10% Hit Rate	46.8%	76.0%	78.4%
Top 10% Lift	2.46x	1.43x	1.58x
Top 50% Hit Rate	27.5%	67.5%	65.9%
Success Capture (Top 10%)	24.6%	14.3%	15.8%

1. *THE MODEL WORKS BEST WHEN YOU NEED IT MOST*

During the crisis period when only 19% of people were subscribing, our model identified a group where 47% said yes—more than double the random rate. It’s like finding gold nuggets in a desert. During recovery when half of people were already subscribing, the model still boosted success to 76-78%—turning good odds into near certainty.

2. *TARGETING THE RIGHT PEOPLE DRAMATICALLY BOOSTS RESULTS*

If you call 100 random people during a crisis, you’ll get about 19 subscribers. But if you use the model to call the top 10% of prospects, you’ll get 47 subscribers from those same 100 calls. That’s 28 extra subscriptions without making a single additional call.

3. *THE MODEL MAKES MONEY IN ANY ECONOMIC CLIMATE*

Running the numbers: Let's suppose each call costs about \$10 and each successful subscription brings \$500 profit. Using the model instead of random calling adds approximately \$14,000 extra profit per 100 calls during the crisis (Epoch 1), \$11,500 extra profit per 100 calls during early recovery (Epoch 2), and \$14,000 extra profit per 100 calls in late recovery (Epoch 3). These gains come from the model's ability to identify the most promising prospects turning 19 random successes into 47 targeted successes when success rates are low, and boosting already good success rates from 53% to 76% when economic conditions are favorable. Across all economic conditions, the model consistently delivers substantial value, proving that data-driven targeting pays for itself many times over whether the economy is booming or busting.

4. *HOW TO TARGET BASED ON OUR GOALS*

Want maximum efficiency? Target the top 10-15% of prospects. You'll get the highest success rates and best return on every call dollar. Want maximum volume? Target the top 40-50%. You'll capture 70-80% of all possible subscribers, though with slightly lower efficiency. Want balance? Target the top 25-30%. You'll get strong results without leaving too many potential subscribers on the table.

5. *WATCH THE ECONOMY TO GUIDE YOUR STRATEGY*

When Euribor rates drop below 1% (like during our Epochs 2-3), success rates naturally rise above 50%. During these times, you can confidently call more people. When rates are higher and success rates lower, rely heavily on the model to find the few willing customers.
